Language Models Trained With Helpers Stop Learning, and Forget, What the Helpers Know: Kamin Blocking and a One-Line Fix

Jungyun Han [1]

Abstract

When a language model is trained together with a helper whose predictions enter its likelihood (a copy head, a cache, a frozen expert, a tool, or a retrieved passage), it stops learning, and even forgets, what the helper supplies. We show why: the gradient of a mixture likelihood is the shared prediction error of the Rescorla–Wagner rule, so this is Kamin blocking, and the gate follows the replicator equation. The theory is quantitative: lock-in time is exponential in the vocabulary size, a helper invades only if it is more reliable than the model, and released Copy-Paste LMs sit at the predicted equilibrium (to 10^{-4} at 1.7B). In models we train and in released models up to 1.7B parameters, the model learns exactly the complement of what the helper supplies, loses skills it already had, and leaves facts unstored. Blocking also acts between circuits: a helper stops induction heads from forming, and two attention heads that share an error race, the first to form excluding the other. The race follows from depth: a circuit is a product of factors, and in a controlled test the same head start is erased, kept or amplified as the parametrisation has one, two or three factors. The remedy is the older Bush–Mosteller rule, Shapley credit for each component: adding the model's own cross-entropy removes blocking and erasure of what the LM head learns in every helper setting we test (a blocked circuit returns half-way at 77M and to 74–103% at 608M), while changing the full model's loss by -0.036 to $+0.013$ nats: a copy-augmented Transformer keeps its roughly twofold token efficiency over a plain one while its LM head stays within 0.03 nats of a plain LM instead of 0.17–0.18 (77M, compute-optimal length, two seeds).

[1]Santa Fe College. Correspondence to: Jungyun Han <58016500@sfcollege.edu>.

Preprint. November 11, 2026.

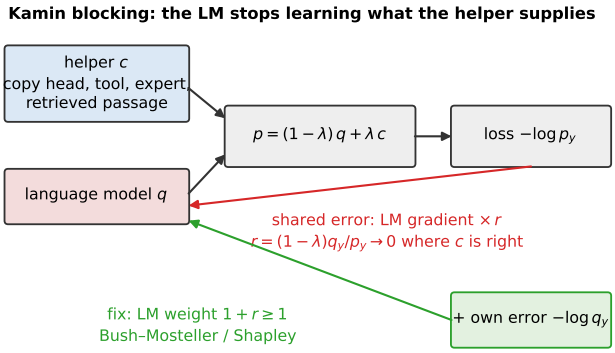


Figure 1. The mechanism and the fix. In a mixture likelihood the LM's gradient is its own cross-entropy gradient scaled by its posterior responsibility r, which vanishes wherever the helper is right: the shared prediction error of the Rescorla–Wagner rule, hence Kamin blocking. Adding the LM's own error restores a gradient weight of at least one on every token.

1. Introduction

Language models are increasingly trained together with helpers that already know part of the answer: copy heads and caches that point into the context (Merity et al., 2017; Grave et al., 2017; Zhong et al., 2022; Godey and Artzi, 2026), frozen expert LMs mixed into the output, tools such as calculators, and passages retrieved into the input. The helper improves the combined prediction. We ask what it does to the model being trained.

In animal learning the answer has been known since Kamin (1969): a cue that is presented together with one that already predicts the outcome acquires little or no association of its own. The Rescorla–Wagner rule (Rescorla and Wagner, 1972) explains this blocking with a prediction error shared by all cues. We show that the likelihood of a model mixed with a helper has exactly this structure (Figure 1), that language models are blocked in the same way—they do not learn, and when the helper is attached later they forget, what the helper supplies—and that the remedy is the older rule in which each cue learns from its own error.

The most striking case is the copy head of Copy-Paste LMs (Godey and Artzi, 2026), which reach a vanilla model's loss up to $2\times$ faster at 1.7B parameters; the gain has been

attributed to a better-trained backbone, but with the copy head removed the LM head is worse than a vanilla model.

The ingredient itself is old: the gradient of a mixture is weighted by each component's posterior responsibility, the E-step of EM (Jacobs et al., 1991). What has not been drawn out is what this does when one component is a model being trained next to a helper that is already competent: the model stops learning, and forgets, exactly what the helper covers, at rates the theory predicts, in the circuits as well as in the loss. This matters whenever the helper is not there or is wrong at use time: a retrieval that misses, a tool that is unavailable, a context that is truncated, or a fine-tuned fact that was always shown in a passage and so was never stored (Section 9). Our contributions:

- **Theory:** the mixture gradient is the Rescorla–Wagner shared error and the gate follows the replicator equation; lock-in time is exponential in the vocabulary size (Thm. 1) and a helper invades only if it is more reliable than the model (Prop. 2); credit is conserved, so a helper blocks the stage it replaces and accelerates the stage it reads (Prop. 5); the fix is Shapley credit.
- **Evidence:** delegation from 30M to 1.7B parameters, graded by per-token helper reliability; complement learning and erasure with tools, copy heads and frozen experts; retrieval that leaves facts unstored in released 135M–1.7B LMs; induction heads that never form; and Kamin's design between two heads of one Transformer, where the head formed first blocks the other only if they share an error, and removing it releases the block.
- **Why circuits race:** a circuit's strength is a product of factors, and under a shared error gradient flow erases a head start for one factor, keeps it for two and amplifies it for three or more (Prop. 6); in controlled runs on the same features only the parametrisation changes, and the three outcomes appear as predicted (two seeds).
- **Fix:** the Bush–Mosteller own-error term, which removes blocking and erasure of what the LM head learns at little cost to the full model (at most $+0.013$ nats; not a race between circuits already lost) and lets a copy-augmented Transformer keep its roughly $2\times$ token efficiency over a plain one while its LM head stays a usable LM (at most 0.03 nats behind a plain one instead of 0.17–0.18 at compute-optimal length, 77M, two seeds).

2. Setup

Our models. Pre-norm GPT with RMSNorm, RoPE, QK-norm, ReLU^2 MLP, untied embeddings and logit soft-cap 15, trained on FineWeb-edu (GPT-2 BPE) with AdamW and a cosine schedule. Small: d=256, 6 layers, 30M or 120M tokens. Medium: d=512, 8 layers, 77M parameters, 100M or 300M tokens. Large: d=1024, 16 layers, 305M parameters, 100M tokens. The learned copy head uses 4 low-rank (rank 64) similarity heads between the final state at t and the final state at $j-1$, sums pointer mass per token, and mixes with a sigmoid gate: $p = (1-\lambda)\, q + \lambda\, c$.

parameters	30M	77M	163M	305M
plain Transformer	5.137	4.264	4.116	4.023
+ copy head + plain CE (ours)	4.650	3.984	3.920	3.846
difference	-0.49	-0.28	-0.20	-0.18
LM head alone (ours)	4.990	4.193	4.122	4.030

Table 1. Validation loss at matched training tokens (30M tokens for the smallest model, 100M otherwise), mean of two seeds. The copy head costs +3.7% training time with the fused loss. The LM head of our model, used alone, matches or beats the plain Transformer, so the gain is not bought by a weaker language model.

Released models. CPLM-135M/360M (20B tokens) and CPLM-1.7B (200B tokens, plus intermediate checkpoints) with vanilla baselines trained by the same recipe (Godey and Artzi, 2026). Their `no_copy` mode gives the LM head alone (the `<copy>` slot removed and renormalised).

LM head alone. The same weights, evaluated with the copy path removed. We split per-token loss by whether the target already occurs in the context (*seen*, about half of all tokens) or not (*new*).

Reproducibility. Training code, analysis scripts, run commands for every experiment and the training logs are at `https://github.com/hjyact/kamin-blocking` (archived at `https://doi.org/10.5281/zenodo.24124317`); every number in the paper is computed from those logs or from the released checkpoints. Losses within a table come from one evaluation sample; different tables use different samples, so the same model's loss can differ slightly across tables; comparisons are made within a table.

3. A copy-augmented Transformer trained without delegation

We start with the payoff and explain it in the rest of the paper. A copy head trained with the usual mixture likelihood $-\log p_y$ makes its LM head worse than a plain Transformer (Section 4); trained with one extra term, the LM's own cross-entropy ($-\log p_y - \log q_y$, "plain CE" below; Section 7), it does not, and the combined model is better than the plain one at every size we train (Table 1).

Compute-optimal training. At 20 tokens per parameter (77M, 1.5B tokens) the copy-augmented model with the extra term reaches 3.533 and 3.534 (two seeds) against 3.591 and 3.594 for the plain Transformer, about $2\times$ the tokens

model	training	full model	LM head alone	vanilla
ours, 77M	100M tok.	$4.002 \pm .002$	4.381	4.261
ours, 77M	300M tok.	3.751 / 3.755	4.058 / 4.068	3.857
ours, 305M	100M tok.	3.838 / 3.842	4.190 / 4.194	4.023
CPLM-135M	20B tok.	2.769	2.939	2.791
CPLM-360M	20B tok.	2.586	2.715	2.604
CPLM-1.7B	200B tok.	2.223	2.309	2.229

Table 2. Validation loss (nats/token). Pairs are two seeds. The LM head trained inside the mixture is worse than a vanilla LM at every scale; for the released models its loss exceeds the copy head's net gain several times over.

	full model		LM head alone	
model	seen	new	seen	new
vanilla (ours, 77M)	2.865	5.866	2.865	5.866
copy head	2.415	5.841	3.204	5.712
copy head, stop-gradient into backbone	2.544	5.936	3.224	5.780
copy head + plain CE (ours)	2.404	5.809	2.861	5.706
CPLM-1.7B, vanilla	–	–	1.390	3.335
CPLM-1.7B	1.375	3.341	1.561	3.294

Table 3. Seen/new split. The LM head gives up tokens already in the context and is better on new ones: a division of labour.

by a three-point fit; the gain comes from the copy head (without the term the full model reaches 3.528), and what the term buys is the LM head: 0.02–0.03 behind the plain Transformer alone, against 0.17–0.18 without it (two seeds; Appendix H).

Alternatives and compute. A copy head on a frozen LM gains 0.09 nats instead of 0.28, and helper dropout is worse on both outputs (Appendix H); the plain Transformer needs about 2.1×, 1.7× and 1.6× the tokens (77M, 163M, 305M) to match the copy-augmented model at 100M.

4. The LM head delegates copying

Removing the copy path exposes what the LM head learned inside the mixture. In every model we train and in the released Copy-Paste LMs it is worse than a vanilla LM of the same recipe (Table 2), and only on tokens that already occur in the context; on new tokens it is better (Table 3): the LM hands copying to the helper. A 608M model trained for this test shows the same, as predicted before the run: its LM head alone is 0.203 and 0.209 nats behind a vanilla LM (two seeds), all of it on tokens already in the context (Appendix H.4).

Graded blocking in a 1.7B model. Rescorla–Wagner predicts that a cue is blocked in proportion to how well the other cue already predicts the outcome. We bin the tokens of 120 validation documents by the released CPLM-1.7B copy head's own probability of the target, c_y (renormalised over the context), and compare its LM head alone with the vanilla model trained by the same recipe, at three points of the 200B-token run. (Figure 2c; per-checkpoint values in Table 17, Appendix I.) The same curve holds for the released 135M and 360M models (20B tokens; Table 16, Appendix I); its amplitude falls with size and training length (top bin 0.79, 0.50, 0.33), and the gain where the copy head cannot help grows slightly (-0.047 to -0.057).

All three copy mechanisms we trained delegate (77M, 100M tokens, LM head alone vs. vanilla 4.261): exact-token cache 4.416, continuous cache (Grave et al., 2017) 4.421, learned pointer 4.382. In the released 1.7B run the delegation is in place at the first checkpoint and decays slowly ($+0.089$ at 42B to $+0.074$ at 200B tokens) while the copy head's net gain stays near 0.02 (Figure 5).

5. Why: responsibility-weighted gradients

With $q = \mathrm{softmax}(z)$ and $p = (1-\lambda)q + \lambda c$, for target y

$$\frac{\partial(-\log p_y)}{\partial z_v} = r\left(q_v - \mathbf{1}[v=y]\right), \quad r = \tfrac{(1-\lambda)\,q_y}{p_y} \in [0,1]. \tag{1}$$

The LM head receives the plain cross-entropy gradient scaled by its posterior responsibility r, the E-step weight of a two-component mixture. Three predictions follow. (i) Where the copy head explains y, $r \to 0$: the LM stops learning copyable tokens while every other position's normaliser keeps pushing them down, so the loss rises on *seen* tokens only (Table 3). (ii) r depends on the copy head's output, not on backpropagation through it, so blocking that gradient does not prevent delegation (LM alone 4.422 vs. 4.381). (iii) Early in training c is diffuse and $r \approx 1$; delegation appears once the copy head is sharp (small model: -0.065 at 30M tokens, $+0.186$ at 120M).

The stage rule. A parameter that feeds the helper also receives $-r_c \nabla \log c_y$, with $r_c = \lambda c_y / p_y = 1 - r$: what the helper takes from the stage parallel to it, it gives to the stage it reads, so a trusted helper blocks the writing it replaces and accelerates the matching it reads (Proposition 5, Section 9).

How strong the blocking is. For a helper that is right on a token ($c_y = 1$) with gate λ, the LM's gradient is its plain gradient times

$$r = \frac{(1-\lambda)\,q_y}{(1-\lambda)\,q_y + \lambda} \;\le\; \frac{1-\lambda}{\lambda}\,q_y. \tag{2}$$

An untrained LM has $q_y \approx 1/V$, so once the gate trusts the helper ($\lambda \ge 1/2$) the LM learns that token at most $1/V$ as fast as alone—2×10^{-5} for $V = 50{,}304$, and $1/100$ even for the 100-value answers of our synthetic tasks. Blocking is therefore all but complete whenever the helper becomes competent before the LM does, and absent when the LM is competent first ($q_y \approx 1$ gives $r \approx 1-\lambda$, and the gate has no reason to move). Retrieval in the context has no gate, but

shows the analogous order dependence (Section 9). Once the LM is blocked, nothing in the loss holds its knowledge in place, so a skill it already had is free to drift (erasure). The extra term restores a gradient of $1 + r \geq 1$ regardless of λ.

Lock-in is exponential in the vocabulary size. The bound above is per step; integrated over training with a learned gate it becomes a statement about time.

Theorem 1. *Consider one context with target y, LM probability $q = e^z/(e^z + K)$ for a trainable logit z and fixed $K > 0$, a helper that is always right ($c_y = 1$), gate $\lambda = \sigma(g)$, and gradient flow on $\ell = -\log\big((1-\lambda)q + \lambda\big)$ in (z, g). Suppose that at some time the gate has reached $\lambda_0 \geq \frac{1}{2}$ while $q_0 \leq \frac{1}{4}$, and let $u_0 = 1 - \lambda_0$. Then the time until q doubles is at least*

$$t_{2\times} \;\geq\; \frac{2}{\lambda_0^2 u_0}\Big(2^{\lambda_0^3/(4q_0)} - 1\Big).$$

Trained alone, or with the extra term $-\log q$, q instead reaches $\frac{1}{2}$ within time $2\log\big((1-q_0)/q_0\big)$.

Corollary. With the extra term at weight $\beta > 0$, $\dot{z} \geq \beta(1-q)$, so q reaches $\frac{1}{2}$ within $2\log((1-q_0)/q_0)/\beta$: any weight turns exponential lock-in into a time $\propto \log V/\beta$. With a perfect, trusted helper and SGD on a 64-way task (Appendix C) the steps to 50% accuracy are 18,000, 6,000, 2,000, 750 and 250 for $\beta = 0.001$, 0.003, 0.01, 0.03, 0.1: $\beta t \approx 18$–25, the predicted $1/\beta$ law over two decades.

For an untrained LM $q_0 \approx 1/V$: once the gate trusts the helper, learning the token takes time exponential in the vocabulary size instead of logarithmic, a fixed point for practical purposes (proof in Appendix D).

6. Mixture training is an evolutionary game

The gate's gradient has a form familiar from population biology. With $\lambda = \sigma(g)$ and $p = (1-\lambda)q + \lambda c$,

$$\dot{\lambda} = \lambda^2(1-\lambda)^2\,\frac{c_y - q_y}{p_y} = \lambda(1-\lambda)\cdot\lambda\Big(\frac{c_y}{p_y} - 1\Big),$$

which is the replicator equation for two strategies whose fitnesses are their shares of the target's probability (c_y/p_y and q_y/p_y, with mean fitness 1), up to the time change $\lambda(1-\lambda)$. Gate learning is selection between the LM and the helper; LM learning is the evolution of one strategy under that selection. Delegation is competitive exclusion, the complement result is niche partitioning, and the lock-in of Theorem 1 is an evolutionarily stable state of the helper. Bayesian updating of fixed hypotheses is already known to follow the replicator equation (Harper, 2009; Shalizi, 2009); what is new here is that the strategies are themselves learners whose competence the selection controls, which is what produces blocking, erasure and lock-in. Population-genetic tools then make predictions.

Proposition 2 (Invasion)**.** *Let the helper be right on a token with probability ρ, independently of anything the gate can condition on, and put its mass elsewhere otherwise, and let the LM give the target probability q. At $\lambda \to 0$ the expected gate gradient has the sign of $\rho(1-q) - q(1-\rho)$: the helper invades if and only if $\rho > q$, and $\lambda = 0$ is stable when $q \geq \rho$. For a perfect helper ($\rho = 1$) the excluding term vanishes and the helper invades any LM with $q < 1$.*

The proof is in Appendix B.

Test. Attaching the partial calculator of Section 9 with reliability ρ to an LM pretrained on the covered problems (three seeds, two competences), a perfect helper invades in all six runs, and imperfect ones are excluded in 21 of 24 (three invade partly, gates 0.40–0.66; Table 22): the proposition describes stability at attachment, not every race. From scratch, an imperfect helper first blocks and is then overtaken, later the more reliable it is (within a factor of two of $350/(1-\rho)$ for $\rho \leq 0.9$; Figure 2a); at $\rho = 1$ never, as Theorem 1 quantifies. Lock-in is a singular limit at perfect reliability, and delegation grows with the helper's per-token reliability (Figure 2c).

Released models sit at the replicator equilibrium. Since $r_c - \lambda = \lambda(1-\lambda)(c_y - q_y)/p_y$, a stationary gate equals the helper's mean responsibility. Binned by their effective gate, held-out tokens of the released Copy-Paste LMs give mean gate vs. mean responsibility 0.1203 vs. 0.1219, 0.1088 vs. 0.1098 and 0.0950 vs. 0.0951 at 135M, 360M and 1.7B, every bin within 0.01 (Table 8): the gates have converged, so the delegation is the game's equilibrium, and the gate predicts the LM head's deficit per token (Appendix G.5).

7. The fix

We train with $\mathcal{L} = -\log p_y - \log q_y$. The LM head's gradient weight becomes $1 + r \geq 1$ on every token, while the copy head and gate still learn from the mixture. The term is Shapley credit (Shapley, 1953): with coalition values $v(S) = \log p_S(y)$, the LM's Shapley value $\frac{1}{2}[v(\{\mathrm{LM}\}) - v(\emptyset)] + \frac{1}{2}[v(\{\mathrm{LM}, H\}) - v(\{H\})]$ has gradient $\frac{1}{2}(\nabla \log q_y + \nabla \log p_y)$, while plain mixture training follows $\nabla_i[v(N) - v(N \setminus i)]$, each component's Vickrey–Clarke–Groves marginal contribution (Clarke, 1971), which is zero wherever another component already explains the token: blocking. The weight must not depend on the gate: the gate-weighted own errors of mixtures of experts (Jacobs et al., 1991) vanish once the gate trusts the helper and leave the calculator task blocked (0–7% against 76–100%, three seeds; Table 21). With several trainable components every one needs its own term:

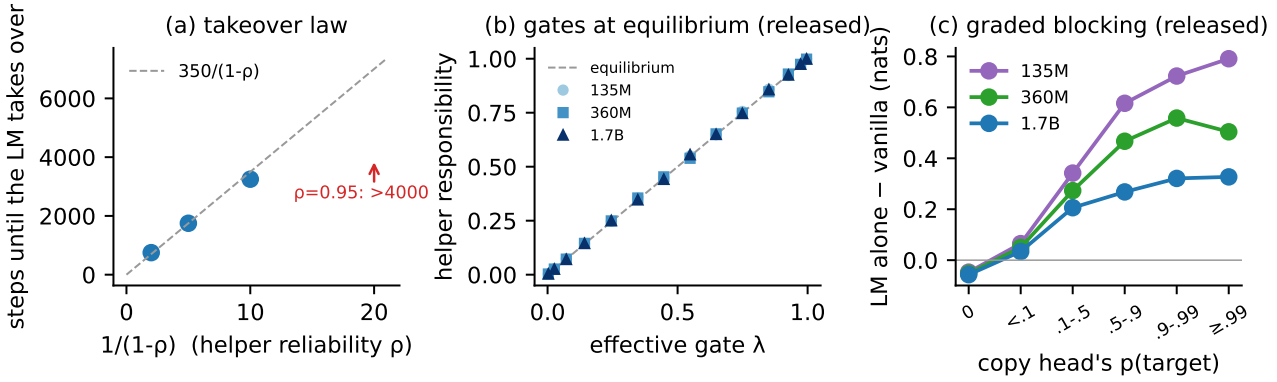


Figure 2. (a) An imperfect helper is overtaken later the more reliable it is (one seed shown; about $350/(1-\rho)$ steps here, within a factor of two across three seeds for $\rho \leq 0.9$); at $\rho = 1$ never. (b) Released Copy-Paste LMs sit at the replicator equilibrium: binned by the gate, the helper's mean responsibility equals the gate at 135M, 360M and 1.7B (Table 8). (c) In released Copy-Paste LMs delegation rises with the copy head's per-token reliability, and the LM head beats a vanilla model where copying cannot help.

with a frozen helper and a second trainable one, fixing only the LM leaves the second helper dead (1–7% alone; Appendix G.14; cf. Jeffares et al., 2023). Order alone also helps: because keeping a circuit needs less credit than forming it, delegating only after the induction onset keeps the circuit and gains 0.009–0.013 nats at 77M and 0.008–0.014 at 305M (two seeds each) without any extra term (Appendix G.13).

Across every scale and both training lengths we train from scratch, the extra term changes the full model's loss by -0.034 to $+0.013$ nats (means of two seeds; better in five of eight settings) and brings the LM head alone from 0.12–0.23 nats behind a vanilla LM of the same size and data (seven of eight settings; the smallest is already ahead) to between 0.15 ahead and 0.04 behind it; at 608M it brings the LM head from 0.20–0.21 behind to within 0.005 and improves the full model (3.783–3.787 against 3.811–3.819, two seeds; Appendix H.4) (Table 11, Appendix H). Its reach is mainly the LM head: at 77M it brings a blocked circuit back only half-way, at 608M to 74–103% of the vanilla model's (0.343–0.415 against 0.404–0.461, and 0.089–0.127 without it, two seeds; Appendix H.4) (Appendix G.7) and does not undo a race between circuits already lost (Appendix G.9).

8. Beyond copy heads: delegation is a property of mixture training

Section 5 never uses the fact that the helper copies. Mixed with a frozen small Transformer (loss 5.145; learned gate), a 77M LM trained for 100M tokens ends at 9.29 / 9.13 nats alone against 4.26 trained alone, and even an expert far weaker than the trained LM (6.28) locks it in, because at initialisation the LM is worse than any trained expert and r starts near zero; the extra term restores 4.25–4.28 (Table 13). Delegation needs only a mixture likelihood and a helper that explains some tokens better than the model being trained.

9. Blocking, complement and erasure: tools, released LMs and retrieval

The mechanism needs only a main model and an auxiliary that already explains part of the data. We therefore train a small Transformer (4 layers, d=256) on sequences of two-digit additions "a+b=c;", mixed with a calculator that puts all its mass on the correct next digit of each answer (a learned gate decides when to defer to it). Trained mixed with the calculator, the LM alone answers 1.7%, 0% and 0% of digits (three seeds) against 100% trained alone; with the extra term 100%, 100% and 75.7% (Table 21, Appendix I).

The LM learns the complement of the tool. Rescorla–Wagner predicts that a blocked cue acquires exactly what the blocking cue fails to predict (Section 11). We test this with a calculator that answers only problems without a units carry (55% of them) and is silent on the rest. The task and the skill are the same everywhere, and the covered problems are the easier ones. (Table 20, Appendix I: covered 33/34/38%, uncovered 100%, three seeds; with the extra term 100%.) The LM does not lose an abstract ability: it adds perfectly where the tool is silent and stays at 33–38% where it is right. Its competence is the complement of the helper's: removing the covered answers from the loss with no tool at all gives the same 32–36% (Proposition 7). The fix removes the hole.

A tool erases a skill the LM already has. Pretrained alone to 100% and continued on the same data mixed with

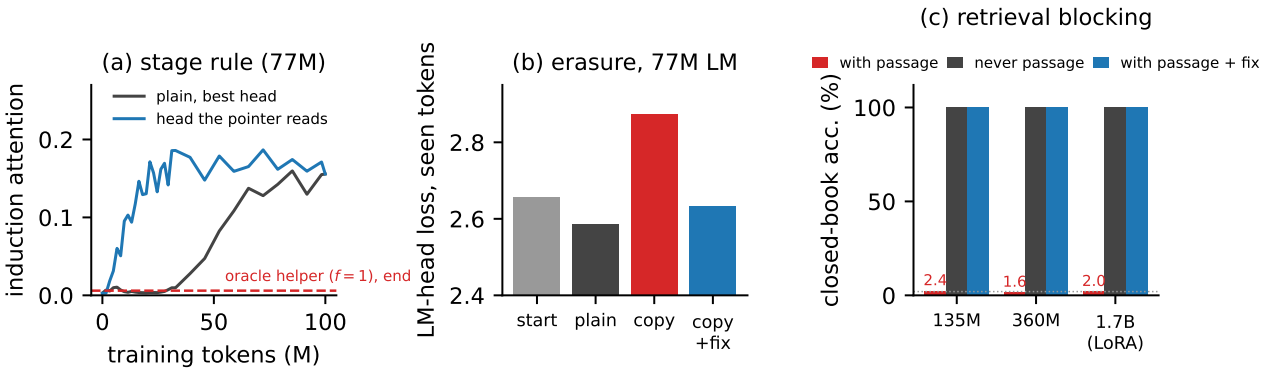


Figure 3. (a) Stage rule: the head a pointer helper reads becomes an induction head about seven times sooner (onset step 182 against 1,300; 194 against 1,432 at a second seed); an oracle helper leaves none. (b) Erasure in a 77M LM continued with a copy head: its LM head ends worse than it started on tokens seen in context; the extra term prevents it. (c) Released LMs fine-tuned on facts always given in a passage stay at chance (dotted, 2%) closed-book; facts never given a passage, or given one with the extra term, are learned perfectly.

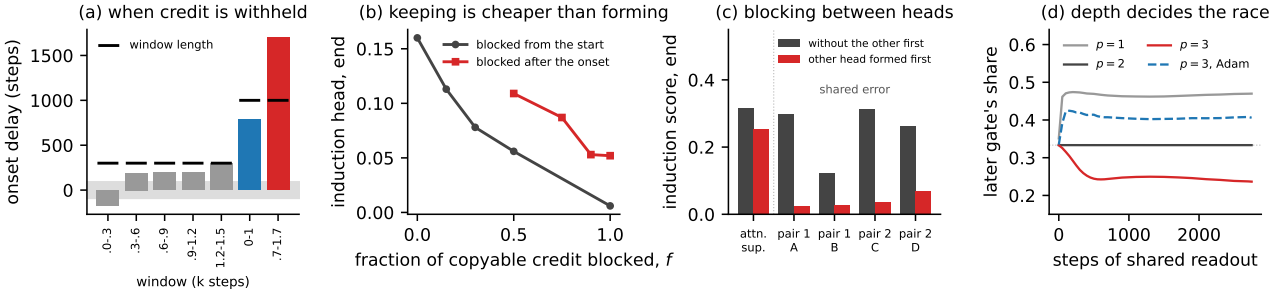


Figure 4. Circuit formation under withheld credit (77M, 100M tokens). (a) Delay of the induction onset when all copyable credit is withheld for a window, against a baseline of $1{,}300 \pm 90$ steps (grey band); ticks mark the window length. (b) Induction head at the end against the blocked fraction, blocked from the start or from step 1,600. (c) Final induction score of a head without (dark) and with (red) another head formed first; supervising attention directly blocks little, a shared error blocks almost completely. (d) Two gates reading the same head under one shared error, the first starting twice as large (dotted line: the starting share); only the number of factors per gate (or the optimiser) differs: one factor erases the head start, two keep it exactly, three amplify it for good, and Adam erases it.

the partial calculator, the LM falls to 34–36% on the covered problems on all three seeds and keeps 100% on the rest; with the extra term it keeps 100% (Table 24; details and the dependence on step size in Appendix I).

The same erasure in a language model. A plain 77M LM continued for another 100M tokens with the copy head attached loses what it knew on tokens already in the context: its LM head ends at 2.87 nats there against 2.66 at the start and 2.59 for plain continuation (two seeds), and the extra term keeps it at 2.63 (Table 23). Released models show the same: instruction tuning with the copy head lowers the CPLM-1.7B LM head's copying gain (3.45 $\rightarrow$ 2.79) while its vanilla twin's rises, and a released 135M LM continued with a frozen 360M expert ends worse than it started (2.866 against 2.791; 2.787 with the extra term; Tables 9 and 19).

Blocking stops a circuit from forming. Shared error predicts more than a worse loss: the circuit that solves the covered positions should not be trained by them. We give a plain 77M LM an oracle helper (ungated product, $1 - 10^{-2}$ on the target) on a fraction f of the induction-copyable positions, those whose bigram (x_t, y_t) occurred earlier. The LM alone then loses in-context copying there and nowhere else (2.09 to 3.64 nats for $f = 0 \rightarrow 1$; other positions unchanged), and its best induction head falls from 0.153 to 0.006 at $f = 1$ (two seeds; 0.228 to 0.007 at 305M), while the previous-token heads, the circuit's first stage that other positions also use, remain. Our learned copy head does the same at 77M (best induction head 0.007) and weakens the circuit at 305M (0.166 against 0.264). The converse also holds (Proposition 5): a helper that reads a backbone head's matching makes that head an induction head about seven times sooner (step 182 against 1,300, and 194 against 1,432 at a second seed), and the released Copy-Paste LMs, whose copy head reads the backbone's matching, keep the matching and lose the writing (Table 9). Section 10 follows the circuits further.

Retrieval blocking in a released LM. A pretrained LM already reads answers from its context, which meets Kamin's condition (Table 25). Fine-tuned on 500 fictional people born in one of 50 cities, with a passage stating the fact before every training question for half of them, released LMs stay at chance closed-book on that half (2.4%, 1.6%, 2.0% at 135M, 360M, 1.7B with LoRA; chance 2%) and learn the other half perfectly; adding each question closed-book gives 100%, and blocking survives distractors and paraphrases (Table 18, Appendix I).

Demonstrations block the task itself. A helper the model grows works the same way: fine-tuned with labelled demonstrations before every question, SmolLM-135M and 360M read the label from them (100%) and learn the task without them only partly (50–82% against 96–98%); the in-weights skill freezes within 50 steps, and demonstrations in half the examples or training without them first prevent it (Appendix I.2).

10. Circuits compete for one error

The shared error also governs circuits inside one Transformer: when a circuit forms, what keeps it, and how two circuits compete. Runs are the plain 77M LM (100M tokens) unless stated; the induction score is a head's attention to the successor of an earlier occurrence of the current token on natural text. Identical configurations differ by about 90 steps in onset and 0.01–0.02 in final score.

Circuits form on a clock of credited data. Withholding a fraction f of the copyable credit delays the induction onset as $1/(1-f)$, and the clock counts credited data, not optimiser distance: halving or doubling the learning rate moves the onset by under 12% in two seeds (1,449 and 1,223 against 1,300 steps; 1,268 and 1,327 against 1,432), where a learning-rate clock predicts a factor of two; at 305M the onset even moves the other way. When credit is withheld matters: the same 1,000-step block costs 785 and 587 steps (two seeds) at the start and 1,696 and more than 1,620 (no onset within training) during formation, when what has been built erodes (Figure 4a).

Keeping a circuit is cheaper than forming it. With half the credit from step 1,600 an existing induction head holds and grows (0.111, two seeds), while half the credit from the start reaches only 0.03–0.06 (305M: 0.158 and 0.187 against 0.047 and 0.086, two seeds) (Figure 4b). The hysteresis gives a second remedy: delegating to the copy head only after the onset keeps the LM's circuit and improves the full model by 0.009–0.013 nats at 77M and 0.008–0.014 at 305M (two seeds each) with no extra term.

Kamin's design between two heads. We pull one head toward induction for the first 300 steps (a small auxiliary loss), then read two heads' summed outputs together, so that they share one error. The head formed first keeps the other from becoming an induction head (0.02–0.07 against 0.10–0.32 without the head start, two pairs, two seeds; at 305M 0.045–0.058 against 0.18–0.21), and the blocked models copy no worse, so the second head was redundant. Without the head start the race can still be lost to noise (one of four runs of one pair). Supervising both heads' attention directly, with no shared error, lets both form (Figure 4c). The model's own loss is such a shared error: at 77M a head placed early keeps the other induction heads of ordinary training from forming, and removing it releases them within 300–600 steps (two seeds), after which all coexist, so competition acts on formation, not on circuits that exist (at 305M, with 256 heads, nothing is blocked).

Why it is a race: depth. Blocking between circuits is not the graded share of linear Rescorla–Wagner; it switches between a head start of 0.053 and one of 0.094. A circuit's strength is a product of factors, and for a product of p factors gradient flow on a shared error erases a head start for $p = 1$, keeps it for $p = 2$ and amplifies it for $p \geq 3$ (Proposition 6). We test this with everything else fixed: fifteen gate pairs in one run read the same two heads' detached outputs, and differ only in how many scalar factors each gate has. The head start is erased, kept and amplified as predicted (amplification exponents 0.05–0.09, 0.74–0.84 and 1.16–1.42 for $p = 1, 2, 3$, two seeds); when both gates read one head, a 2:1 head start ends at 1.1:1, 2:1 to three decimals, and 3.1–3.2:1 (Figure 4d). Adam erases it (1.5:1), and why Adam-trained heads still race we could not explain (six tests, Appendix G.10); the race also explains why a larger outcome does not unblock and the own-error term does not undo a lost race. Details, controls and every failed prediction are in Appendices G.8, G.9 and G.10.

11. Related work

Copy and pointer mechanisms (Merity et al., 2017; Grave et al., 2017; Zhong et al., 2022; Godey and Artzi, 2026) and retrieval-augmented LMs (Lewis et al., 2020; Borgeaud et al., 2022) are evaluated with the helper on; we ask what the trained model knows without it, and Godey and Artzi (2026) attribute the CPLM gain to the LM head's gradient bottleneck (Godey and Artzi, 2026a). Tool use is usually learned after pretraining (Schick et al., 2023); our tool results concern a helper present in the training likelihood. Responsibility weighting drives expert specialisation in mixtures (Jacobs et al., 1991; Shazeer et al., 2017), and products of experts (Hinton, 2002) are our ungated case; related failures are gradient starvation (Pezeshki et al., 2021), shortcut learning (Geirhos et al., 2020) and collapsed gates (Fang et al., 2026); blocking is used on purpose in debiasing (Clark et al., 2019) and gradient routing (Cloud et

phenomenon	prediction for a model trained with a helper	result	where
blocking	a trusted helper stops the model learning what it supplies	LM at chance; 1.7B fact recall at 2%	§9
overshadowing	two trained components split the targets	LM .29–.39 / helper .36–.43 / overlap .03–.06	App. G.14
complement learning	the model learns exactly what the helper leaves	33–38% = masked training 32–36%	App. I
backward blocking	revising the helper after joint training changes the model; sign set by the coupling	Kalman -1.59 to -1.60; shared trunk opposite sign	App. G.3
conditioned inhibition	a product-trained model pushes down helper-boosted context tokens	partly: -0.43 and -0.45 against the plain model (two seeds), product only; never below absent tokens	App. H
overexpectation	two calibrated members trained jointly in a product both lose confidence	.71 $\rightarrow$ .29–.30 (both members)	App. G.15
superconditioning	a pessimistic helper ($-m$) raises the model's margin by m	$+1.00$–1.05, $+1.99$–2.04, $+3.00$–3.05 for $m = 1, 2, 3$	App. G.15
blocking between circuits	of two heads sharing one error, the one formed first keeps the other from forming	0.02–0.07 against 0.10–0.32, two pairs, two seeds; also at 305M; none without a shared error	App. G.9
release from blocking	removing the blocking head lets the blocked ones form	best other 0.079 $\rightarrow$ 0.336 within 600 steps (77M, two seeds); at 305M nothing is blocked to release	App. G.9
pre-exposure (latent inhibition)	a head first trained to another pattern loses the race	fails: a useless pattern is dropped (0.297 against 0.312); only a role the circuit needs keeps the head out (0.024); one seed separates, a second does not	App. G.9
unblocking by a larger outcome	a larger shared error lets the blocked head form, a smaller one does not (Pearce–Hall: both do)	fails: neither does (0.013–0.095 against 0.009–0.028; two pairs, two seeds in one); the extra error goes to the winner	App. G.9
own-error rule (Bush–Mosteller)	each component's own term removes blocking	every helper setting; not a race between circuits already lost	§7, 10

Table 4. Classical predictions of the Rescorla–Wagner rule, translated to components trained in one likelihood and tested; the synthetic cells (overshadowing, backward blocking, overexpectation, superconditioning) are ranges over three seeds, the others as stated in the appendix. Eight hold, two fail, one holds in part.

al., 2024). Induction heads form in an abrupt phase change (Olsson et al., 2022; Reddy, 2024), whose depth-dependent, rich-get-richer learning dynamics we use to explain the race between them (Saxe et al., 2014; Woodworth et al., 2020) and compete with in-weights strategies (Singh et al., 2025); we show what sets their onset and how they compete for one error. Our product case is the Rescorla–Wagner rule (Rescorla and Wagner, 1972), itself the delta rule of adaptive networks (Sutton and Barto, 1981), in logit space, the mixture case its responsibility-scaled form, and the own-error rule of Bush and Mosteller (1951) the fix. Of eleven classical predictions, four of them between circuits inside one Transformer, eight hold, two fail, one in part (Table 4; more in Appendix F).

Limitations. Our training runs reach 608M parameters (two seeds at every size from 77M) and at most 1.5B tokens; most circuit results are 77M runs (two seeds; the main ones also at 305M); larger-scale claims rest on released models, which we analyse but do not retrain. The fix restores what the LM head learns, not a race between circuits already lost, and several of our predictions failed; Appendix A lists them. The memorisation reading of retrieval blocking is empirical, not a privacy guarantee.

12. Conclusion

A component trained inside a likelihood that a helper explains learns only what the helper leaves. This is Kamin blocking, and it follows from the shared error of a mixture likelihood alone: with copy heads, caches, frozen experts, tools, retrieval and demonstrations, in models up to 1.7B, and between attention heads that share an error, where circuits form on a clock of credited data and race because they are products of factors. One own-error term, Bush and Mosteller's rule and Shapley credit for the components, restores what the model learns; delegating after a circuit forms is a second remedy. Helpers are worth training with, provided each component is also paid for its own predictions.

Impact Statement

This paper explains why language models trained with helpers (copy heads, caches, frozen experts, tools, retrieval) stop learning what the helper supplies, and gives a one-line training fix. A positive consequence is more efficient and more self-sufficient models; a model that relies on a helper it is later deprived of may fail silently, and our fix reduces that risk. Blocking also has a privacy reading: facts that are always supplied in context are not stored, which could be used to keep data out of a model's weights, but the effect is empirical and not a guarantee (Appendix A). We see no other direct societal risks beyond those of language-model training in general.

References

ai4privacy. PII-masking-200k dataset. https://huggingface.co/datasets/ai4privacy/pii-masking-200k, 2024.

E. Akyürek, B. Wang, Y. Kim, J. Andreas. In-context language learning: architectures and algorithms. ICML 2024.

S. Amari. Integration of stochastic models by minimizing α-divergence. Neural Computation 19(10):2780–2796, 2007.

S. Arora, N. Cohen, E. Hazan. On the optimization of deep networks: implicit acceleration by overparameterization. ICML 2018.

S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. ICML 2022.

R. R. Bush and F. Mosteller. A mathematical model for simple learning. *Psychological Review*, 58(5):313–323, 1951.

N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song. The secret sharer: evaluating and testing unintended memorization in neural networks. In *USENIX Security*, 2019.

C. Clark, M. Yatskar, and L. Zettlemoyer. Don't take the easy way out: ensemble based methods for avoiding known dataset biases. In *EMNLP-IJCNLP*, 2019.

E. H. Clarke. Multipart pricing of public goods. Public Choice 11:17–33, 1971.

A. Cloud et al. Gradient routing: masking gradients to localize computation in neural networks. arXiv, 2024.

P. Dayan, S. Kakade. Explaining away in weight space. NIPS 13, 451–457, 2000.

A. Deshpande, K. Narasimhan. Guiding attention for self-supervised learning with Transformers. Findings of EMNLP 2020.

D. M. Dwyer, N. J. Mackintosh, R. A. Boakes. Simultaneous activation of the representations of absent cues results in the strengthening of an excitatory association between them. JEP: Animal Behavior Processes 24:163–171, 1998.

Y. Fang, Z. Zhang, X. Wang. When adaptation fails: a gradient-based diagnosis of collapsed gating in vision-language prompt learning. arXiv:2605.09549, 2026.

R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann. Shortcut learning in deep neural networks. Nature Machine Intelligence 2:665–673, 2020.

N. Godey, Y. Artzi. Copy-Paste Language Models. arXiv:2610.19247, 2026.

N. Godey, Y. Artzi. Lost in backpropagation: the LM head is a gradient bottleneck. arXiv:2603.10145, 2026.

E. Grave, A. Joulin, N. Usunier. Improving neural language models with a continuous cache. ICLR 2017.

T. Groves. Incentives in teams. Econometrica 41(4):617–631, 1973.

A. Hans et al. Be like a goldfish, don't memorize! Mitigating memorization in generative LLMs. arXiv:2406.10209, 2024.

M. Harper. The replicator equation as an inference dynamic. arXiv:0911.1763, 2009.

H. He, S. Zha, and H. Wang. Unlearn dataset bias in natural language inference by fitting the residual. In *DeepLo Workshop*, 2019.

G. E. Hinton. Training products of experts by minimizing contrastive divergence. Neural Computation 14(8):1771–1800, 2002.

R. Jacobs, M. Jordan, S. Nowlan, G. Hinton. Adaptive mixtures of local experts. Neural Computation, 1991.

A. Jeffares, T. Liu, J. Crabbé, M. van der Schaar. Joint training of deep ensembles fails due to learner collusion. NeurIPS 2023.

L. J. Kamin. Predictability, surprise, attention, and conditioning. In *Punishment and Aversive Behavior*, 1969.

D. C. Krakauer. Competitive and complementary tools. arXiv:2607.18460, 2026.

E. F. Kremer. The Rescorla–Wagner model: losses in associative strength in compound conditioned stimuli. Journal of Experimental Psychology: Animal Behavior Processes 4(1):22–36, 1978.

K. Kujanpää, P. Marttinen, H. Valpola, and A. Ilin. Efficient knowledge injection in LLMs via self-distillation. arXiv:2412.14964, 2024.

F. Kunstner, A. Milligan, R. Yadav, M. Schmidt, A. Bietti. Heavy-tailed class imbalance and why Adam outperforms gradient descent on language models. NeurIPS 2024.

P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

A. Lv, R. Xie, X. Sun, Z. Kang, R. Yan. Language models "grok" to copy. arXiv:2409.09281, 2024.

N. J. Mackintosh. A theory of attention: variations in the associability of stimuli with reinforcement. *Psychological Review*, 82(4):276–298, 1975.

S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer sentinel mixture models. ICLR 2017.

C. Olsson, N. Elhage, N. Nanda, et al. In-context learning and induction heads. arXiv:2209.11895, 2022.

O. Ovadia, M. Brief, M. Mishaeli, and O. Elisha. Fine-tuning or retrieval? Comparing knowledge injection in LLMs. In *EMNLP*, 2024.

I. P. Pavlov. *Conditioned Reflexes*. Oxford University Press, 1927.

J. M. Pearce, G. Hall. A model for Pavlovian learning: variations in the effectiveness of conditioned but not of unconditioned stimuli. Psychological Review 87:532–552, 1980.

M. Pezeshki, S.-O. Kaba, Y. Bengio, A. Courville, D. Precup, G. Lajoie. Gradient starvation: a learning proclivity in neural networks. NeurIPS 2021.

G. Reddy. The mechanistic basis of data dependence and abrupt learning in an in-context classification task. ICLR 2024 (arXiv:2312.03002).

R. A. Rescorla and A. R. Wagner. A theory of Pavlovian conditioning: variations in the effectiveness of reinforcement and nonreinforcement. In *Classical Conditioning II*, 1972.

R. A. Rescorla. Reduction in the effectiveness of reinforcement after prior excitatory conditioning. Learning and Motivation 1(4):372–381, 1970.

A. M. Saxe, J. L. McClelland, S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. ICLR 2014.

T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom. Toolformer: language models can teach themselves to use tools. arXiv:2302.04761, 2023.

A. See, P. Liu, C. Manning. Get to the point: summarization with pointer-generator networks. ACL 2017.

C. R. Shalizi. Dynamics of Bayesian updating with dependent data and misspecified models. *Electronic Journal of Statistics*, 3:1039–1074, 2009.

D. R. Shanks. Forward and backward blocking in human contingency judgement. Quarterly Journal of Experimental Psychology 37B:1–21, 1985.

L. S. Shapley. A value for n-person games. In Contributions to the Theory of Games II, 307–317, 1953.

N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. ICLR 2017.

A. K. Singh, T. Moskovitz, S. Dragutinovic, F. Hill, S. C. Y. Chan, A. M. Saxe. Strategy coopetition explains the emergence and transience of in-context learning. ICML 2025.

R. S. Sutton, A. G. Barto. Toward a modern theory of adaptive networks: expectation and prediction. Psychological Review 88(2):135–170, 1981.

W. Vickrey. Counterspeculation, auctions, and competitive sealed tenders. Journal of Finance 16(1):8–37, 1961.

G. Wang, D. Murfet. Patterning: the dual of interpretability. arXiv:2601.13548, 2026.

B. Woodworth, S. Gunasekar, J. D. Lee, E. Moroshko, P. Savarese, I. Golan, D. Soudry, N. Srebro. Kernel and rich regimes in overparametrized models. COLT 2020.

D. Wurgaft, E. S. Lubana, C. F. Park, H. Tanaka, G. Reddy, N. D. Goodman. In-context learning strategies emerge rationally. arXiv:2506.17859, 2025.

Z. Yang, Z. Dai, R. Salakhutdinov, W. Cohen. Breaking the softmax bottleneck: a high-rank RNN language model. ICLR 2018.

Z. Zhong, T. Lei, D. Chen. Training language models with memory augmentation. EMNLP 2022.

X. Zhou et al. Making harmful behaviors unlearnable for large language models. In *Findings of ACL*, 2024.

J. Zhu, J. Li, Y. Hao, C. Zou, R. Fu, M. Yang. Learning less is more: premature upper-layer attention specialization hurts language model pretraining. arXiv:2605.10504, 2026.

Overview. Where each main-text claim is supported, and on how many runs.

claim (section)	evidence	where
the LM head delegates copying (§4)	77M–608M, two seeds; released 135M–1.7B	Tab. 2, App. H
lock-in time, invasion, equilibrium (§5, 6)	proofs; 64-way task over two decades of β; released gates	App. D, B, G.5
the fix and why its weight must not depend on the gate (§7)	every scale, two seeds; calculator, three seeds	App. H, I
complement learning and erasure (§9)	calculator three seeds; 77M LM two seeds; released 135M	App. I
retrieval blocking (§9)	released 135M, 360M, 1.7B	App. I
a helper stops circuits forming (§9)	77M two seeds; 305M	App. G.4, G.7
credit clock and hysteresis (§10)	77M and 305M, two seeds for hysteresis	App. G.8
blocking and release between heads (§10)	two pairs, two seeds; 305M one seed per arm	App. G.9
depth decides the race (§10)	proposition; fifteen gate pairs per run, two seeds	App. G.10
classical predictions (Table 4)	synthetic cells three seeds; circuit cells as stated	App. G.1
failed predictions	all listed	App. A

A. Limitations

We train from scratch only up to 608M parameters (two seeds; Appendix H.4) and 300M tokens; the 1.7B evidence comes from released checkpoints. The copy head's net gain over a plain Transformer shrinks with training (77M: -0.26 at 100M tokens, -0.11 at 300M; released 1.7B: -0.006 to -0.02 at 200B), so our scaling table should be read as a short-training result; at 77M the gain persists at compute-optimal length (-0.058 at 1.5B tokens, one seed), but we have not trained larger models that long. At 300M tokens the gain also shrinks with model size (77M: -0.11; 305M: 3.528 vs. 3.603, -0.075; LM head alone 3.630). Without the fix the 305M / 300M-token copy head gains only -0.046 (3.557, both seeds) and its LM head alone is 0.233 worse than the plain Transformer (3.836): part of the shrinking gain is delegation, and the fix keeps 1.6× more of it at the largest scale we train. Post-hoc repair recovers only part of the gap and saturates; we have not retrained a 1.7B model with the fix from scratch. Downstream evaluation covers one copy-heavy and one copy-light task. The calculator and retrieval tasks are small synthetic ones (4-layer models, 2,000 facts). The released-LM experiments are short continuations (8M tokens or 1,200 steps), mostly one run each (the expert case has two data slices), on fictional facts with a fixed question template; with the fix, transfer to a paraphrased question is limited (36–39%), as for facts fine-tuned without passages. The correspondence to Kamin blocking is at the level of the learning rule (shared versus own error); we do not claim that the networks implement associative learning in any other sense. The circuit results (oracle and pointer helpers, the stage rule's tests, the scaffold), the remaining Shapley and optimiser experiments are one run per configuration (the overexpectation, superconditioning, backward-blocking and overshadowing tests of Table 4 were repeated over three seeds), at 77M parameters and 100M tokens or on synthetic tasks; the released-model circuit measurements use one set of eight repeated random sequences. Several predictions failed and are reported as such (time rescaling of induction blocking, progressive erasure of the writing in the 1.7B run, stage-local repair, the scaffold at 300M tokens, responsibility-split Adam in a language model, the strong form of conditioned inhibition, the escape-time estimate across combination rules, erasure by instruction tuning in the full-attention CPLM, a lag reading of the gate's small offset during training, a pure credit clock for withheld credit, a weaker credit law in the optimiser-limited regime, active suppression by false-copy positions, a sharp upkeep threshold, the null slot as a tax shelter, the gradient ledger as a predictor of interventions and of which head wins, collapse of a later head under supervised attention, a graded (linear Rescorla–Wagner) dose–response of blocking, a ratio law for the race, latent inhibition between heads, two validation readings of single pulled runs, the own-error fix undoing blocking between heads, unblocking by a larger outcome, a below-0.005 gain from delegation order at 305M (the gain was 0.008–0.014), graded sharing between one-layer induction heads, Adam with other signals restoring the depth race, freeing a blocked head by removing its competing objectives, freeing it by halving the gradient noise, a race of bounded gates under Adam, and freeing a blocked head by removing weight decay), and some explanations were found only after the results (the $(1-f)^{0.6}$ dependence, the depth account of the race (written after the first upshift run), the non-copy gain of the extra term, the $-\log(1-\lambda)$ deficit law). The memorisation results are empirical, not a guarantee: they cover 135M and 360M models, up to 400 records, three non-adaptive attacks and one adaptive one, and an overfitting regime; unlike differential privacy, blocking bounds nothing formally, and a helper that is absent or wrong for some occurrences of a secret

would let those occurrences be stored.

B. Proof of Proposition 2

Proof. When the helper is right, $\partial_g(-\ell) = \lambda(1-\lambda)(1-q)/\big((1-\lambda)q+\lambda\big)$; when it is wrong, $c_y = 0$ and $\partial_g(-\ell) = -\lambda(1-\lambda)q/\big((1-\lambda)q\big) = -\lambda$. The expectation is $\lambda(1-\lambda)\big[\rho(1-q)/((1-\lambda)q+\lambda) - (1-\rho)/(1-\lambda)\big]$, whose sign at $\lambda \to 0$ is that of $\rho(1-q)/q - (1-\rho)$. □

C. Lock-in, vocabulary size and the optimiser

Theorem 1 is for gradient flow. We test it on a minimal task: 512 keys, each mapped to one of K labels, an embedding and a two-layer MLP, and a perfect helper whose gate starts trusted ($\lambda_0 = 0.95$); one run per cell, 20k steps.

LM-alone accuracy after 20k steps	$K=2$	4	8	16	64	256
alone (SGD)	100%	100%	100%	100%	100%	100%
mixed with a perfect helper, SGD	54%	30%	22%	10%	1%	0%
mixed with a perfect helper, Adam	98%	97%	96%	92%	86%	91%

Table 5. Under SGD the LM stays at or near chance ($1/K$) for every K, with slight progress only for small vocabularies, as the theorem predicts. Adam largely undoes the blocking here.

The Adam row has a simple reading: in this task every parameter receives only blocked gradient, and Adam's per-coordinate normalisation rescales it to full size. In the calculator, the copy-head LMs and the released models, blocked and unblocked tokens share parameters, so the normaliser is set by the large unblocked gradients and the blocked ones stay small. We tested this reading by letting the helper cover only half of the keys ($K = 64$): under Adam the covered keys then stay at 3–16% over 20k steps while the uncovered ones reach 100% (SGD: 3% and 100%), against 86% when every key is covered. Blocking under Adam needs blocked and unblocked signals to share parameters, as they do in every language model we study.

Proposition 3 (Normalised updates)**.** *Consider sign descent, $\dot\theta = -\eta\,\mathrm{sign}(\nabla\ell)$, the limit of Adam without moment averaging. (a) If a logit z receives only the blocked gradient $r\,(q-1)$ of Theorem 1, then $\dot z = \eta$ whatever $r > 0$, and q reaches $\frac{1}{2}$ within $\log((1-q_0)/q_0)/\eta$: logarithmic in V, not exponential. (b) If a coordinate receives $r\,g_b + g_u$ from a blocked and an unblocked token, its update is $-\eta\,\mathrm{sign}(g_u)$ whenever $r|g_b| < |g_u|$, so the blocked token moves that coordinate only through the unblocked token's sign.*

Proof. (a) $\mathrm{sign}(r(q-1)) = -1$ for $r > 0$, $q < 1$, so $\dot z = \eta$ and $\dot q = \eta q(1-q)$; integrate. (b) $\mathrm{sign}(r g_b + g_u) = \mathrm{sign}(g_u)$ when $|r g_b| < |g_u|$. □

Case (a) is the Adam row of Table 5, where every parameter sees only blocked keys; case (b) is the half-covered task, the calculator and the language models, where the normaliser is set by unblocked gradients. With moment averaging the same holds in expectation: Adam divides by $\sqrt{\mathbb{E}[g^2]}$, which unblocked tokens dominate. Case (b) predicts a late escape: once the unblocked keys are learned their gradient keeps shrinking, and when it falls below the blocked one the shared coordinates follow the blocked keys. With half the keys covered and the gate trusted ($K = 64$, Adam, one run), the uncovered keys reach 100% by step 2,000 while the covered ones stay at 3–5% until step 14,000 and then rise (11%, 20%, 34% at 18k, 22k, 30k) to a plateau of 31–37% up to 100k, as the gate's growing trust shrinks r in turn; under SGD the covered keys stay at 3–4% for 100k steps. Labels are random per key, so the rise is not transfer. Logging the two gradients on the shared MLP confirms the mechanism: the unblocked gradient keeps falling after its keys are learned ($\|g_u\| = 0.46$ at step 0, 2.2×10^{-6} at 10k, 2.4×10^{-7} at 14k) while the blocked one stays near 5×10^{-8}, and the share of coordinates where the blocked gradient is the larger one goes from 1.5–2.6% (10k–12k) to 11.5% at 14k and 31.5% at 16k, exactly when the covered keys start to rise. Case (b) also suggests a fix at the optimiser rather than the loss: normalise the gradient from covered and from uncovered positions with separate moment estimates. On the half-covered task this "responsibility-split" Adam brings the covered keys to 93% by step 5,000 (plain Adam: 3%). It does not transfer to a language model: in the 77M copy-augmented LM (100M tokens, split by whether the target occurs in the context) the full model ends at 5.301 and the LM alone at 5.904, against 4.095 and 4.504 for the same optimiser with a random split of equal sizes and about 4.00 and 4.386 for plain Adam. It does not diverge: both runs start at 9.29, and the split run stalls from about step 200

(training loss 5.63 at 400 and 1,000 steps against 5.03 and 4.51), mainly on new tokens, as if the two equally normalised updates cancelled on shared parameters; we did not measure that, and we do not propose the method. **A geometric reading.** Blocking scales the mixture's Fisher information along the blocked tokens' directions by r^2, so the loss is nearly flat exactly where the LM should learn. Natural gradient multiplies those directions by F^{-1} and should therefore undo blocking, but only in directions that unblocked data do not share. We test this with an exact, damped natural gradient on a linear readout over fixed features (64 keys, 8 labels, half covered by a perfect helper with a fixed gate of 0.95, one run). Mean probability of the target after 3,000 steps, covered / uncovered keys:

	keys with their own feature directions	keys sharing an 8-dimensional feature space
gradient descent	0.144 / 0.956	0.234 / 0.501
natural gradient	**0.994** / 1.000	0.229 / 0.757

Table 6. Natural gradient removes blocking where the blocked keys own their directions and does not where they share them with unblocked keys, as the Fisher-degeneracy reading predicts. Adam, a diagonal approximation, behaves like the left column when no parameter receives an unblocked signal and like the right one otherwise.

We also predicted that Muon, which orthogonalises each matrix update and so equalises its singular directions, would rescue the covered keys even with shared parameters. It does not: 19% with half the keys covered (97% with all covered), the same pattern as Adam.

D. Proof of Theorem 1

Write $u = 1 - \lambda$ and $p = (1-\lambda)q + \lambda = 1 - u(1-q)$. With $dq/dz = q(1-q)$ and $d\lambda/dg = \lambda u$,

$$\dot{z} = -\partial_z \ell = \frac{u\,q(1-q)}{p}, \qquad \dot{g} = -\partial_g \ell = \frac{\lambda u(1-q)}{p},$$

so $\dot{\lambda} = \lambda u\,\dot{g} = \lambda^2 u^2 (1-q)/p \geq 0$: the gate never decreases, hence $\lambda \geq \lambda_0$ and $p \geq \lambda \geq \lambda_0$ from time 0 on. Then

$$\frac{d}{dt}\log q = (1-q)\,\dot{z} = \frac{u\,q(1-q)^2}{p} \leq \frac{u\,q}{\lambda_0}, \qquad \dot{u} = -\frac{\lambda^2 u^2 (1-q)}{p} \leq -\lambda_0^2 u^2 (1-q).$$

Let τ be the first time with $q = 2q_0$. On $[0, \tau]$, $q \leq 2q_0 \leq \frac{1}{2}$, so $\dot{u} \leq -\frac{\lambda_0^2}{2} u^2$, which gives $u_t \leq \left(1/u_0 + \lambda_0^2 t/2\right)^{-1}$, and

$$\log \frac{q_t}{q_0} \leq \frac{2q_0}{\lambda_0} \int_0^t \frac{ds}{1/u_0 + \lambda_0^2 s/2} = \frac{4q_0}{\lambda_0^3} \log\left(1 + \frac{\lambda_0^2 u_0 t}{2}\right).$$

At $t = \tau$ the left side equals $\log 2$, so $1 + \lambda_0^2 u_0 \tau/2 \geq 2^{\lambda_0^3/(4q_0)}$, which is the claim. Alone, $\dot{z} = 1 - q \geq \frac{1}{2}$ while $q \leq \frac{1}{2}$, so z rises by $\log\left((1-q_0)/q_0\right)$—enough to reach $q = \frac{1}{2}$—within twice that time; with the extra term $\dot{z}$ is larger still. □

The theorem is for gradient flow on one token with all other logits fixed; our experiments use Adam, which rescales small gradients, and shared parameters, through which blocked tokens can still drift (erasure). We have not extended the bound to these settings; the experiments in Sections 9 and 7 are the evidence there.

E. A Price decomposition of mixture training

In population genetics the Price equation splits the change in mean fitness into selection (shifts in composition) and transmission (change within types). For $p = (1-\lambda)q + \lambda c$ the analogue is exact: $\Delta p = (1-\bar{\lambda})\Delta q + \bar{\lambda}\Delta c + \overline{(c-q)}\Delta\lambda$ with midpoint averages, and dividing by the logarithmic mean of p and p' splits $\Delta \log p$ into LM transmission, copy transmission and selection. Logging (λ, q, c) per token on a fixed batch every 250 steps of the 77M copy-head runs (100M tokens) gives the cumulative split (the three terms sum to the measured change to three decimals):

cumulative change in $\log p$	LM transmission	copy transmission	selection
original loss, tokens seen in context (55%)	$+3.947$	$+0.554$	$+0.575$
original loss, new tokens (45%)	$+5.343$	0	-0.275
with the extra term, seen	$+3.991$	$+0.548$	$+0.517$
with the extra term, new	$+5.379$	0	-0.318

Table 7. About 11% of the gain on copyable tokens is selection, and the gate costs about 0.3 nats on new tokens by moving mass to the copy head where it cannot help. The total change is dominated by leaving the random initialisation, so this split separates the two training losses only weakly; the LM head's standalone quality (Table 11) is a different quantity.

F. Extended related work

Copy and pointer mechanisms for language modelling (Merity et al., 2017; Grave et al., 2017; Zhong et al., 2022; Godey and Artzi, 2026) and summarisation (See et al., 2017); hard-wired n-gram heads (Akyürek et al., 2024). Godey and Artzi (2026) attribute the CPLM gain to the gradient bottleneck of the LM head (Godey and Artzi, 2026a) and support it with a stop-gradient ablation; they evaluate models with the copy head on. Responsibility weighting is the mechanism behind expert specialisation in mixtures of experts (Jacobs et al., 1991); we show it produces an unwanted specialisation of the LM head. Jacobs et al. (1991) also contrast a shared (cooperative) error with per-expert errors weighted by the gate; our fix differs in giving the LM its own error at a weight that does not depend on the gate, and the gate-weighted version does not remove blocking (Table 21). Related failure modes are known elsewhere: gradient starvation (Pezeshki et al., 2021), shortcut learning (Geirhos et al., 2020), expert collapse in mixtures of experts, and collapsed gates between a frozen backbone and trainable prompts in vision-language prompt learning (Fang et al., 2026). We identify the mechanism in language-model pretraining with copy and expert mixtures, measure it in released checkpoints and downstream, and give a fix that does not change the architecture. Load-balancing losses for mixtures of experts (Shazeer et al., 2017) equalise how often the gate routes to each expert; they act on routing frequency, not on each component's own likelihood, and here the gate should route to the helper wherever it is right, so balancing targets a different quantity from the one that blocking starves. The competition between in-context and in-weights learning, in which context-constrained in-weights strategies replace in-context learning (Singh et al., 2025) and strategies are selected as a Bayesian posterior (Wurgaft et al., 2025), is the same selection inside one network, with the context as the helper; retrieval blocking (Section 9) is its case of a perfectly reliable context. Shortcut learning describes a model preferring an easy cue inside one network; blocking is the analogous preference across two components that share one error signal, with the consequence that the starved component, not the shortcut, is what is deployed alone.

The same structure is used deliberately in dataset debiasing: a robust model is trained in a product or mixture with a frozen bias-only model so that it learns only what the bias cannot explain (Clark et al., 2019; He et al., 2019). Security vectors (Zhou et al., 2024) make harmful behaviours unlearnable in fine-tuning by attaching a frozen module that already predicts the harmful data, and gradient routing (Cloud et al., 2024) localises what is learned by masking gradients. That is blocking put to use, and Theorem 1 quantifies why it is so effective; our results show the same mechanism acting unintentionally whenever a helper that the model should also learn from enters the likelihood, and that the model then loses, not only fails to gain, what the helper covers.

Knowledge-injection work finds that fine-tuning teaches new facts poorly compared with retrieval (Ovadia et al., 2024) and injects them by distilling a teacher that sees the document into a student that does not (Kujanpää et al., 2024); our retrieval result gives a reason for the first and a minimal version of the second (train each question closed-book as well).

G. Mechanisms: credit, circuits and equilibrium

This appendix collects the results that go beyond the copy head: the classical associative-learning predictions (G.1), combination rules and credit for any α (G.2), retrospective revaluation (G.3), blocking of induction circuits and the stage rule (G.4), the gate's equilibrium in released models (G.5), released models' circuits, the scaffold (G.12), Shapley credit (G.14), and overexpectation and superconditioning (G.15).

G.1. The associative-learning predictions, tested

Each classical phenomenon below follows from the shared error of the Rescorla–Wagner rule (or, for backward blocking, from a learner with posterior covariance); we translated each into a prediction for components trained in one likelihood

and tested it. One run per cell unless stated. (Table 4 in Section 11.)

G.2. Any combination rule: selection locks in, sharing slows down

We replace the mixture by Amari's α-integration (Amari, 2007) with the same learned gate, $m_\alpha \propto [(1-\lambda)q^t + \lambda c^t]^{1/t}$, $t = (1-\alpha)/2$ ($\alpha = -1$ is the mixture, $\alpha \to 1$ the normalised geometric mean, i.e. a gated product of experts), and by an ungated logit sum $p \propto q\,c$. Task of Appendix C ($K = 64$, SGD, 20k steps); the helper puts $1-\varepsilon$ on the right label.

LM-alone accuracy	$\alpha = -1$	$\alpha = 0$	$\alpha \to 1$	ungated sum
gate starts untrusted ($\lambda_0 = 0.05$)	4%	7%	14%	57%
gate starts trusted ($\lambda_0 = 0.95$)	1%	1%	1%	57%
trusted, with the extra term	100%	100%	100%	100%

Every gated rule locks in, and the gate goes to the helper ($\lambda \geq 0.99$) in all of them, so the lock-in of Theorem 1 comes from selection (the gate), not from the algebra of the mixture; the geometric rule blocks less from an untrusted start, in the order of t. Without a gate there is no lock-in, but the solo learning signal is the residual error of the combined prediction, which a near-perfect helper sets to $O(\varepsilon)$: this is the Rescorla–Wagner rule itself. The prediction is that the learning time scales as $1/\varepsilon$, and it does: the LM-alone accuracy reaches 57% after 2k steps at $\varepsilon = 10^{-2}$, after 20k at $\varepsilon = 10^{-3}$, and is 4% after 20k at $\varepsilon = 10^{-4}$. The two mechanisms therefore separate: a learned gate turns blocking into a lock-in, and shared error alone turns it into a slow-down whose rate is the helper's error rate. The extra term removes both.

Proposition 4 (Credit for any α)**.** *Let $m(v) \propto [\sum_i w_i\, p_i(v)^t]^{1/t}$ with $t = (1-\alpha)/2 > 0$, normalised over v, and let component i have parameters θ_i. Then $\nabla_{\theta_i} \log m(y) = r_i(y)\, \nabla_{\theta_i} \log p_i(y) - \mathbb{E}_{v\sim m}[r_i(v) \nabla_{\theta_i} \log p_i(v)]$ with $r_i(v) = w_i p_i(v)^t / \sum_j w_j p_j(v)^t$ and $\sum_i r_i = 1$. As $t \to 0$ (a gated product) $r_i \to w_i$.*

Proof. Write

$$\log m(v) = \frac{1}{t} \log \sum_j w_j p_j(v)^t - \log Z;$$

differentiating the first term gives $\frac{1}{t} \cdot w_i t\, p_i^{t-1} \nabla p_i / \sum_j w_j p_j^t = r_i \nabla \log p_i$, and $\nabla \log Z = \mathbb{E}_m[\nabla \log \tilde{m}]$. □

Credit is therefore conserved under every combination rule, and the stage rule (Proposition 5) holds for all of them. The rules differ in what sets the shares: for $t = 1$ a component's share grows with its own probability of the target, so a correct helper takes the credit wherever it is right; as $t \to 0$ the share is the gate weight alone, whatever the components predict, so an ungated product never locks in and only slows down through the shared residual, while a gated one locks in through the gate. This is the split between selection and sharing measured above. From $r_q \approx \frac{1-\lambda}{\lambda} q^t$ for a correct helper we predicted that, with the gate fixed, the escape time would scale as K^t/t, so that smaller t escapes sooner. It is the other way round: with $\lambda = 0.9$, SGD and $K = 64$, the mixture ($t = 1$) reaches 50% after 13,250 steps (87% at 40k), while $t = 0.75$, 0.5 and 0.25 stay at 9%, 3% and 1% after 40k steps. The approximation drops the normaliser term, which grows as t falls because the helper's small mass on wrong labels is raised to the power t; the proposition holds, our escape-time estimate from it does not. Keeping the normaliser, the one-logit model of Theorem 1 ($K = 64$, $\varepsilon = 10^{-3}$, $\lambda = 0.9$, gradient descent with step 0.5) gives initial gradients of 1.7×10^{-3}, 1.0×10^{-3}, 4.9×10^{-4} and 2.6×10^{-4} and escape times of 1,294, 2,266, 5,808 and 18,267 steps for $t = 1, 0.75, 0.5, 0.25$: the measured order, though the network's ratios are larger than this one-logit model's.

G.3. Retrospective revaluation: its sign is set by the coupling

In people, training $AX+$ and then $A+$ alone weakens X after the fact (backward blocking; Shanks, 1985), which Rescorla–Wagner cannot produce and a Kalman learner produces through the negative posterior covariance that joint training creates (Dayan and Kakade, 2001). We test the analogue: joint training of an LM and a helper (logit sum), then the helper alone is trained on half of the keys toward the label ($A+$) or toward uniform ($A-$); the LM never enters phase 2.

LM-alone $\log q_y$ on revalued keys (controls)	$A+$	$A-$
separate parameters, exact Kalman / natural gradient	$-0.39 \to -1.98$ (0)	$-0.39 \to -0.04$ (0)
separate parameters, SGD or diagonal	no change	no change
shared trunk, SGD	$-0.09 \to -0.02$ (-0.02)	$-0.09 \to -1.00$ (-0.20)
shared trunk, Adam	$-0.02 \to -0.00$ (0)	$-0.02 \to -1.52$ (-0.25)

A learner with cross-parameter curvature shows backward blocking and release from overshadowing exactly where the helper was revalued; a gradient learner whose components share a trunk shows the opposite sign, the pattern known in animal learning as mediated learning (Dwyer et al., 1998). After joint training, revising the helper therefore changes what the deployed LM knows, and the direction is set by how the two are coupled, not by the objective; weakening a helper that shares the backbone damages the LM on those tokens beyond the interference seen on control keys.

G.4. An oracle helper prevents in-context copying from forming

We give a plain 77M Transformer (100M tokens) an oracle helper, combined by an ungated product, that puts $1 - \varepsilon$ ($\varepsilon = 10^{-2}$) on the target at a random fraction f of the *induction-copyable* positions (the bigram (x_t, y_t) occurred earlier in the sequence; 16% of training positions) and is uniform elsewhere. By the Rescorla–Wagner rule the LM's logit gradient at a covered position is $O(\varepsilon/V)$, so the circuit that solves exactly these positions should not be trained by them. We report the LM alone on validation text, split by position type.

oracle coverage f	0	0.5	0.75	0.9	1	1, with the extra term
loss, induction-copyable positions	2.09	2.64	2.99	3.19	3.64	2.66
loss, other positions	4.71	4.67	4.69	4.69	4.68	4.68
copying gain over $f = 1$ (fraction of $f = 0$)	1.55	1.00 (65%)	0.65 (42%)	0.45 (29%)	0	0.98 (63%)

The helper removes in-context copying from the LM where it covers it and nowhere else: the other positions are unchanged. Our prediction that coverage only rescales time ($t^* \propto 1/(1-f)$) holds at $f = 0.5$ (2.64 against 2.66 for the control at half the steps) but fails above it: the gain falls as roughly $(1-f)^{0.6}$, far slower than time rescaling predicts, which is consistent with the parts of the circuit that uncovered positions train being learned at full rate. We test this reading on the attention patterns of saved $f = 0$ and $f = 1$ runs (a sequence of 256 random tokens repeated once): the best induction head (attending from t to the token after the previous occurrence) scores 0.153 in the control and 0.006 with the oracle, while previous-token heads, the first stage of the circuit, remain (best 0.42 against 0.36; layer 1 0.34 against 0.36, although heads in layers 3–6 weaken from 0.30–0.42 to 0.16–0.30). Blocking removes the last stage of the circuit, the one whose output the helper replaces, and leaves the stage that other positions also use. The 77M copy-augmented model of Section 4 (100M tokens, no fix) has no induction head either (best 0.007; previous-token 0.41) and its LM head gains 0.18 nats from the repetition against 1.44 for the plain model and 0.25 for $f = 1$: its copy head computes its own match from projected hidden states, so it replaces the whole circuit. To test the stage rule causally we change only where the match is computed: the copy distribution becomes the attention of head 0 in the last layer, $c(v) = \sum_j a_{tj}[x_j = v]$, mixed by a learned gate (`-attnptr`). That head becomes the model's only induction head (0.181; the other heads of its layer ≤ 0.004), while the LM head's copying gain is 0.45 nats and its loss on induction-copyable positions 2.87; with the extra term the head stays an induction head (0.147), the copying gain is 1.14, the copyable loss 2.12 against 2.09 for the plain model, and the LM alone reaches 4.198 against 4.275. Matching that the helper reads is kept and writing that it replaces is blocked, the pattern of the released Copy-Paste LMs (Table 9). Proposition 7 also says that blocking can only take what the LM could have learned. The released sliding-window variants (window 256 in every layer, CPLM-1.7B and its vanilla baseline) let us test this, since a windowed backbone copies less the further back the previous occurrence is. On 150 held-out documents the LM head's deficit by distance to the previous occurrence of the target is $+0.148$, $+0.208$, $+0.204$, $+0.181$ and $+0.025$ nats for 0–16, 16–64, 64–256, 256–512 and 512–1100 tokens, where the vanilla model's advantage over its new-token level is 2.31, 2.11, 1.68, 1.04 and 0.63; on new tokens the deficit is -0.055. The deficit vanishes where the vanilla model barely copies. The helper's trust falls much less: measured in the same documents, the copy head's mean responsibility is 0.196, 0.211, 0.204, 0.178 and 0.143 in these bins (gate 0.166 to 0.128), so from 256–512 to 512–1100 tokens the deficit falls by 85% while the responsibility falls by 20%: most of the drop is that there is nothing left to take, as the proposition says. The deficit is not proportional to the vanilla model's advantage at shorter distances (it is flat up to 512 tokens), which we do not

explain. The released hybrid variant, with a full-attention layer every fourth layer, copies far back somewhat better (vanilla advantage 0.77 instead of 0.63 beyond 512 tokens) and keeps more of a deficit there ($+0.081$ instead of $+0.025$; $+0.145$, $+0.207$, $+0.198$, $+0.139$ in the nearer bins, -0.042 on new tokens): the direction the proposition predicts, but with a small difference in what the vanilla models can do and a smaller hybrid deficit at 256–512 tokens, so not a clean architectural contrast. The pointer helper adds only a gate to the model. On the same validation batches (77M, 100M tokens) the plain Transformer has 4.300; with the pointer helper the full model has 4.022 (LM alone 4.356) and with the extra term 4.040 (LM alone 4.220), against 4.002 (4.198) for our copy head with the extra term: a head the model already has gives most of the copy head's gain.

Proposition 5 (Stage rule)**.** *Let $p = (1-\lambda)q_y + \lambda c_y$ and split the model's parameters into a writing part w that reaches the loss only through q and a matching part m that reaches it through q and, if the helper reads it, through c. With $r_q = (1-\lambda)q_y/p$ and $r_c = \lambda c_y/p$, so that $r_q + r_c = 1$,*

$$\nabla_w(-\log p) = -r_q \nabla_w \log q_y, \qquad \nabla_m(-\log p) = -r_q \nabla_m \log q_y - r_c \nabla_m \log c_y.$$

As the gate comes to trust a correct helper ($r_q \to 0$) the writing stops learning, while the matching keeps the whole credit if the helper reads it ($\nabla_m c_y \neq 0$) and is blocked with the writing if it does not ($\nabla_m c_y = 0$).

Proof. $\nabla(-\log p) = -[(1-\lambda)\nabla q_y + \lambda \nabla c_y]/p$ for parameters other than the gate, and $(1-\lambda)\nabla q_y/p = r_q \nabla \log q_y$, $\lambda \nabla c_y/p = r_c \nabla \log c_y$; w does not enter c. □

The three cases of this section are its three cases: an oracle reads nothing from the model and our own copy head computes its own match, so the whole circuit is blocked; a pointer built from a backbone head and the released copy head read the match, which is kept and accelerated, while the writing is blocked. Fixing the gate instead of learning it sets the responsibilities directly (pointer helper, 77M, 100M tokens, one seed each): with $\lambda = 0.2$ and 0.8 the LM head's copying gain is 0.544 and 0.252 nats and its copyable loss 2.72 and 5.23, the dose-response the writing term predicts. We first predicted that the matching would form faster at larger λ; applied correctly, the proposition says it should not at first, since early in training $q_y \ll c_y$ makes $r_c \approx 1$ for any λ, and indeed the pointer head follows the same course for 200 steps (0.059 and 0.067) before the $\lambda = 0.8$ run, whose LM barely trains (4.351 against 5.499 alone), falls behind (0.1 reached at step 300 and 450; final 0.168 and 0.107).

Credit is conserved. For $p = (1-\lambda)q_y + \lambda c_y$, a parameter that reaches the loss only through q receives $-r_q \nabla \log q_y$ and one that reaches it only through c receives $-r_c \nabla \log c_y$, with $r_q = (1-\lambda)q_y/p$, $r_c = \lambda c_y/p$ and $r_q + r_c = 1$ on every token. What the helper takes from the parallel path it gives to the path that feeds it. At initialisation $q_y \approx 1/V$ while a pointer over the context gives c_y of order the target's frequency in the context, so $r_c \approx 1$: the pointer head should be trained much faster than the plain model's induction circuit, which is reached only through q. We log the induction attention during training (same two runs re-trained with logging, one seed): the pointer head reaches 0.1 at step ≈300 (0.095 at 300, 0.130 at 600), the plain model's best head at step ≈1,850 (0.010 at 1,000, 0.047 at 1,400, 0.137 at 2,000); both end at 0.156. The LM head first benefits (copyable loss 3.99 against 4.27 at step 400) and is then blocked (2.87 against 2.15 at the end). The same identity is blocking for the parallel path and acceleration for the feeding path. The extra term recovers 63% of the loss; it doubles the weight of uncovered positions, and the other positions improve slightly (4.677 against 4.714). One seed per cell.

G.5. The gate at equilibrium

The released models use a sink, so their mixture is $p(y) = (1 - \alpha(1 - a_{\text{sink}}))\, q_y + \alpha\, p_{\text{copy}}(y)$ with α the <copy> probability; the effective gate is $\lambda = \alpha(1 - a_{\text{sink}})$ and the copy head's responsibility $r_c = \alpha p_{\text{copy}}(y)/p(y)$. 200 held-out documents (204,800 tokens):

gate bin	.00–.01	.01–.05	.05–.1	.1–.2	.3–.4	.5–.6	.8–.9	.99–1
1.7B, mean gate	.003	.025	.071	.142	.346	.547	.851	.996
1.7B, mean r_c	.003	.025	.070	.144	.346	.555	.856	.998
360M, mean r_c (gate)	.003 (.003)	.026 (.026)	.072 (.071)	.145 (.142)	.355 (.347)	.539 (.548)	.847 (.851)	.998 (.996)
135M, mean r_c (gate)	.003 (.003)	.027 (.026)	.074 (.072)	.146 (.142)	.355 (.347)	.550 (.548)	.859 (.851)	.998 (.996)

Table 8. Binned by the effective gate, the copy head's mean posterior responsibility equals the gate, the stationarity condition of the replicator dynamics. Overall: 0.1203/0.1219 (135M), 0.1088/0.1098 (360M), 0.0950/0.0951 (1.7B).

The equilibrium holds throughout training, not only at the end. On the released intermediate checkpoints of the 1.7B run (100 documents each) the mean gate and mean responsibility are 0.1033/0.1053, 0.0957/0.0980, 0.0959/0.0968 and 0.0933/0.0945 at 42B, 84B, 126B and 168B tokens (0.0950/0.0951 at 200B): the gate tracks the equilibrium while it moves, as a fast selection variable following slowly changing competences would. At every intermediate checkpoint the responsibility is slightly above the gate; since the gate falls over training, this is not a lag behind a falling equilibrium, and we do not have an explanation (held-out text, on which the LM is less sure, would raise the responsibility, but the gap is smallest at the end). Our first computation used α as the gate, ignoring the sink, and found the gate 22–28% above the responsibility; reading the models' code corrected it. The identity is the M-step of EM for the gate; what it shows here is that a 1.7B model trained for 200B tokens has its delegation at the equilibrium of the selection dynamics, so the delegation measured in Section 4 is the stable outcome of that game rather than a transient. The gate also predicts the LM head's deficit token by token. Per gate bin (120 held-out documents), the LM head's NLL minus the vanilla model's is $+0.006$, $+0.169$, $+0.449$, $+0.668$, $+1.234$, $+2.420$ at 135M and -0.004, $+0.141$, $+0.407$, $+0.536$, $+0.943$, $+1.724$ at 360M for gates of 0–.01, .1–.2, .3–.4, .5–.6, .8–.9 and .99–1. Up to $\lambda \approx 0.5$ this is close to $-\log(1-\lambda)$ (0, 0.16, 0.43, 0.69): the LM head gives the target about $(1-\lambda)$ times the vanilla probability, the weighted maximum-likelihood solution when each token is weighted by the LM's responsibility $1-\lambda$; at larger gates the deficit falls short of it. We derived this reading after seeing the numbers, and then tested a prediction it makes before looking: the weight r_q depends on the target, near $1-\lambda$ for a target the copy head explains and 1 for one it cannot ($c_y = 0$), so within a gate bin the LM head should be worse than the vanilla model on targets that occur in the context and *better* on targets that do not, both more so at larger gates, because the probability withdrawn from copyable targets is renormalised onto the rest. It is: for gates of .1–.2, .4–.5, .6–.7, .8–.9 and .9–.95 the deficit on seen targets is $+0.32$, $+0.80$, $+1.07$, $+1.34$, $+1.43$ (135M; 360M: $+0.29$, $+0.73$, $+0.87$, $+1.04$, $+1.17$) and on new targets -0.09, -0.43, -0.66, -1.42, -2.03 (360M: -0.13, -0.43, -0.81, -1.41, -2.31; the high-gate new-target bins hold 20–60 tokens). This is the niche partition of Figure 2c, token by token. Taken literally, the reading predicts the delegated head from two other models with no parameter: reweight the vanilla distribution by $w(v) = (1-\lambda)q_{\text{van}}(v)/[(1-\lambda)q_{\text{van}}(v) + \lambda c(v)]$ over the whole vocabulary, with CPLM's own gate and copy distribution, and renormalise. On 40 held-out documents this predicts the gain side well (lowest octile of predicted deficit: -0.147 vs. -0.162 actual at 135M, -0.136 vs. -0.155 at 360M) but only about 60% of the loss side (highest octile $+0.80$ vs. $+1.31$; $+0.66$ vs. $+1.07$) and half of the mean deficit (0.079 vs. 0.160; 0.063 vs. 0.121), with a per-token correlation of 0.32. Reweighting accounts for where probability goes. The weight depends on the LM's own probability, and iterating $q \leftarrow q_{\text{van}}\, w(q)/Z$ to self-consistency does not converge to the measured head but runs off towards full delegation (mean deficit 0.079, 0.219, 0.659, 1.528 after 1, 3, 10, 30 iterations at 135M), the lock-in of Theorem 1; the real head (0.160) lies between one and three iterations, as if the positive feedback had been arrested part of the way. What arrests it, transfer through shared parameters (Proposition 7) or the normaliser's push-down on copyable tokens (Section 5, prediction (i)), we cannot separate here. A cruder, context-level version of the reading, written down before measuring, separates the two sides. If every token already in the context is down-weighted by $1-\lambda$ and the rest renormalised, a new target gains $\log Z$ and a seen one loses $-\log(1-\lambda) + \log Z$, with $Z = (1-\lambda)P_c + 1 - P_c$ and P_c the vanilla model's probability mass on context tokens. For new targets this zero-parameter prediction holds over a factor of fifty: -0.042, -0.144, -0.322, -0.639, -1.190, -1.870 predicted against -0.060, -0.197, -0.410, -0.693, -1.119, -1.994 measured for gates of .05–.2 up to .95–1 at 135M (360M alike). For seen targets it falls far short ($+0.18$ predicted against $+0.84$ at gates .4–.6), and $-\log(1-\lambda)$ is closer: the probability withdrawn from the context is renormalised onto new tokens as predicted, but it is withdrawn from the tokens the copy head points to rather than from every context token, so the loss on the copied target is not diluted. The opposite extreme withdraws exactly the helper's share: if the full model is calibrated, the LM head is the residual $q = (q_{\text{van}} - \alpha p_{\text{copy}})/(1-\lambda)$, clipped at zero. This gets the gain side again (lowest decile of predicted deficit -0.187 vs. -0.210 at 135M, -0.172 vs. -0.192 at 360M) but overshoots the loss three-fold (highest decile $+4.64$ vs. $+1.47$; $+4.06$ vs. $+1.21$): the LM head keeps a floor on targets the copy head explains. The measured head therefore lies between uniform withdrawal from the context and full withdrawal of the helper's share, consistent with the arrested feedback above. The reading also gives a training-free partial repair of a released model that needs neither the copy head nor any parameter: add $-\log(1-\lambda)$, with λ the LM head's own <copy> probability, to the logits of tokens already in the context, and renormalise. On 120 held-out documents the LM head alone goes from 2.950 to 2.911 nats at 135M (vanilla 2.788) and from 2.726 to 2.699 at 360M (vanilla 2.607): about a quarter of the gap, with seen targets improving (2.050 $\rightarrow$ 1.930) and new ones returning to the vanilla level (4.094 $\rightarrow$ 4.157; vanilla 4.138). The correction strength is the one the reading gives; we did not tune it. On LAMBADA, where the answer is a word from the passage, it raises the LM head's accuracy from 7.8% to 10.1% (135M) and from 12.2% to 15.0% (360M), against 31.4% and 37.2% for the vanilla models: about a tenth of the gap, too little to replace repair by training. The redistribution also changes what the LM head says when it is wrong. On the 3,832 LAMBADA passages whose answer's first token occurs in the passage, the LM head's

top-1 first token is absent from the passage 47.7% of the time at 135M and 46.2% at 360M, against 23.3% and 21.9% for the vanilla models, and such tokens make up 53% of its errors against 39–40% of the vanilla models' (we predicted this before measuring it; "in the passage" includes common tokens, so the split is coarse). Without its helper, a delegated model does not just miss the word to copy: it more often substitutes one that is not there.

G.6. Released Copy-Paste LMs keep the matching and lose the writing

	135M		360M		1.7B	
	vanilla	CPLM	vanilla	CPLM	vanilla	CPLM
best previous-token head	0.968	0.983	0.978	0.995	0.894	0.944
best induction head	0.971	0.897	0.962	0.895	0.906	0.952
induction heads > 0.3	6	2	10	2	10	2
LM-head copying gain (nats)	10.77	3.17	10.60	3.09	10.26	4.73

Table 9. Backbone attention and LM-head copying on 256 random tokens (ids 1000–6000) repeated once, 8 sequences; copying gain is the LM head's NLL on the first half minus the second (for CPLM renormalised without the copy token). Released final checkpoints of Godey and Artzi (2026).

Their matching also moves later (layers 18–22 against 7–12 in the vanilla models up to 360M); in our models, moving the pointer's read to layer 3 moves the induction head there (Table 10). The strongest matching head is kept (and is stronger at 1.7B), but the number of induction heads falls from 6–10 to 2, and the LM head copies a third to a half as well: the copy head consumes the matching and replaces the writing. Ablation makes the reading causal. Zeroing the output of the strongest induction head drops the CPLM copy head's probability of the target on the repeated half from 0.647 to 0.250 (135M) and from 0.690 to 0.328 (360M), while zeroing a non-induction head of the same layer changes nothing (0.643, 0.684): the copy head reads the backbone's match. We had also predicted that the LM head would barely notice, its writing being gone already; instead its remaining copying gain vanishes (3.17 to -0.42, 3.09 to -0.91), so what is left of it runs through the same head. In the vanilla models the same ablation costs little (10.77 to 8.23, 10.60 to 10.13), as other induction heads take over. The CPLM backbone keeps two induction heads instead of six to ten, and both the helper and the residual LM copying hang on the strongest of them; we first read this as blocking removing redundancy, but a full head scan of the vanilla model (below) shows that its copying of an exact repetition is also a serial chain (single-head drops of its probability of the target from 0.758: 0.685, 0.643, 0.511), so what differs is only that the vanilla model keeps about a third of that probability after the ablation while the CPLM LM head keeps none of its copying. The fragility does not reach ordinary text, though: zeroing the same head raises the full CPLM's held-out loss on seen targets by 0.025 (135M) and 0.015 (360M) nats against 0.025 and 0.009 for the vanilla models with their own strongest head zeroed, so on natural repetitions other routes carry the copying and we make no robustness claim. Scanning every head of the 135M backbone shows why. On the repeated random sequence the copy head depends on a serial chain: zeroing any one of L11h8, L0h7, L19h4 or L18h5 cuts its probability of the target from 0.628 by 0.598, 0.589, 0.386 and 0.300. On held-out text no head matters much: the largest single-head drop is 0.021 from 0.180 (L0h7), the top five heads account for 28% of all drops, and the matching the copy head reads is spread over many heads. "Hangs on one head" is thus a statement about exact repetition, where the circuit is a chain whose every link is necessary, not about natural text; the vanilla model's own scan gives the same picture for exact repetition (L12h8, L8h4, L9h8, L4h6, L0h7 each remove 0.44–0.69 of 0.758), while on text it leans more on early heads (L0h0: 0.146 of 0.396). The scans also place the two circuits at different depths. The vanilla models' chains sit early (135M: L4–L12; 360M: L7h0, L8h10, L9h13, then L15), the CPLM copy head's late (135M: L18–L19 with L0h7 and L11h8; 360M: L18h12, L18h13, L20h5, L22h13 of 32 layers), as the logit lens found the target at layer 10 against 19 and 22. A model that must write the copied token matches early to leave layers for the writing; a backbone whose match is read off its last hidden state by a helper has no writing to do and matches late. We formed this reading on the 135M models and confirmed it on the 360M ones; at 1.7B (24 layers) the shift is much smaller (CPLM chain L8–L13, led by L9h29; vanilla L3–L10, led by L10h3), although on held-out text the 1.7B copy head draws on the last two layers (L22–L23, top-five share 18%) while the vanilla model's copying leans on layer 0 heads.

G.7. Circuits in our 77M models: where the matching forms

In our 77M models (100M tokens) the extra term brings the circuit back only half-way: the plain model has induction heads of 0.153, 0.114 and 0.110 and a copying gain of 1.44 nats (0.78, 0.39, 0.21 with its one, two, three strongest heads zeroed); the copy-augmented model without the term has none (0.007; gain 0.18); with the term it has 0.088 and 0.073

and a gain of 0.72 (0.49, 0.22, 0.16). Our copy head computes its own match, so the matching credit stays split even when the writing is restored, as the stage rule implies; redundancy, measured by the fraction kept after one ablation, is about the same as in the plain model. With the pointer helper, which reads one backbone head, the circuit collapses onto that head, as in the released models: the pointer head is the only induction head (0.147 with the extra term; every other head ≤ 0.006), and zeroing it takes the LM head's copying gain from 1.137 to 0.153 even though the extra term restored it, against 1.436 to 0.776 for the plain model (without the term: 0.453 to 0.126). Credit conservation concentrates the matching on the head the helper reads; the head forms early and the others never become induction heads (a reading we offer after the result). The location is causal too: moving the pointer from head 0 of the last layer to head 0 of layer 3 moves the induction head there. Head L3h0 becomes the only induction head (0.056, 0.116, 0.181, 0.235 at steps 200, 600, 1,000 and the end; every other head ≤ 0.004), stronger than the plain model's best head in layer 7 (0.153), and zeroing it takes the LM head's copying gain from 0.474 to 0.125. The matching forms where the helper reads, whatever the depth, which is the depth shift of the released models produced by hand. Two further positions test the limits. A pointer on head 0 of layer 0 can never match (one layer cannot form an induction circuit), so the helper should never become reliable and, by Proposition 2, never invade: the gate falls from 0.149 to 0.067, the pointer head stays at 0.001, and the LM builds its own induction head in layer 4 (0.231) and ends slightly better than the plain model (copying gain 1.67 against 1.44; 4.228 against 4.278 alone). We also predicted that a pointer on layer 1 would make L1h0 an induction head, with credit flowing upstream to build a previous-token head in layer 0. It did not: layer 0 grew no previous-token head (best 0.04), L1h0 stayed at 0.001, and the LM formed only a weak induction head in layer 5 (0.106; copying gain 0.48 with the gate at 0.08), a partial blocking we do not explain.

layer the pointer reads	none (plain)	0	1	2	3	7
induction score of the pointer head	–	0.001	0.001	0.232	0.235	0.181
best induction head elsewhere	0.153 (L7)	0.231 (L4)	0.106 (L5)	$\leq$0.005	$\leq$0.004	$\leq$0.004
LM head's copying gain (nats)	1.44	1.67	0.48	0.37	0.47	0.45

Table 10. Pointer helper reading head 0 of a given layer (77M, 100M tokens, one run each). From layer 2 on the read head becomes the model's only induction head and the LM head's copying is blocked; at layer 0 the helper cannot match, is not trusted, and nothing is blocked; layer 1 is the exception we do not explain.

Measured at the end of training on induction-copyable positions, the helper's responsibility is 0.154, 0.228, 0.395 and 0.423 for pointers on layers 0, 1, 2 and 7 (0.04–0.08 on other positions): the layer-1 helper does take more credit exactly where copying is learned, which points the right way, but a step from 0.15 to 0.23 is far too small to explain a copying gain falling from 1.67 to 0.48. Logging it during training (both runs repeated), we predicted that during circuit formation the layer-1 helper would take at least twice the layer-0 helper's credit on these positions. It takes 25–35% more (0.70, 0.44, 0.33, 0.28, 0.35 against 0.57, 0.35, 0.26, 0.22, 0.27 at steps 50, 200, 400, 800, 1,000), so that prediction fails; yet the LM's own induction head starts to grow at step 800–1,000 in the layer-0 run (0.019, 0.032, then 0.156 at 2,000) and has reached only 0.046 at 2,000 in the layer-1 run (0.09 at the end; the repeat reproduces the split, 0.18 against 0.09).

G.8. The credit clock: when circuits form

A modest loss of credit delays a threshold-like phase change a long way. We formed this reading after the fact and then tested it with the oracle helper on 30% of the induction-copyable positions, a comparable loss of credit by a different route: we predicted the best induction head at step 2,000 to stay below 0.06 against 0.137 for the plain model, and it is 0.045 (0.011 at 1,400; 0.078 at the end against 0.156). The circuit is far more sensitive than the loss, which moves only from 2.15 to 2.41 nats on these positions; this is why the loss-based dependence on f above looked mild. The onset itself follows the time-rescaling law that failed for the loss. Defining it as the first step at which the best induction head exceeds 0.05, the plain model's onset is step 1,415; we predicted $1{,}415/(1-f)$, i.e. 1,665 and 2,831 for $f = 0.15$ and 0.5, before running them, and measured 1,491 and 2,930 (and 2,134 against 2,022 at $f = 0.3$). The onset is noisy: three runs of one configuration with the same seed and data order give 1,277, 1,404 and 1,223 (non-deterministic kernels only), so the run-to-run spread is about 90 steps and the law holds within it, no better. The clock counts data, not optimiser distance, over a fourfold range of learning rates: at 0.5, 1 and 2 times the default the onset is at 1,449, $1{,}300 \pm 90$ and 1,223, where a clock in units of learning rate would put it at 2,600 and 650 (with the copy-head model, 911 at half the rate against 898). At a quarter of the default it moves to 2,551, where the optimiser, not the data, becomes the limit; this contrasts with reports that a larger learning rate brings in-context copying forward (Lv et al., 2024). A learning rate weighted clock was

already excluded by the f series: it predicts no onset at all within training for $f \geq 0.3$. We guessed that in the optimiser-limited regime the credit share would matter less, since Adam normalises the size of the updates, and predicted an onset within training for $f = 0.5$ at a quarter of the rate; there is none (head 0.009 at the end), as the credit law predicts (about 5,100). At twice the rate, $f = 0.5$ puts the onset at 2,596 against 2,450 predicted. The two clocks compose: roughly $t^* \approx \max(T_{\text{data}},\, c/\eta)/(1-f)$, with the credit share stretching whichever clock is binding. We then predicted the effect of the extra term from the same law: with $f = 1$ and the term at weight 1, copyable positions get gradient weight 1 against 2 elsewhere, half the plain model's relative share, so the circuit should match $f = 0.5$. A plain model with loss weight 2 on positions that are not induction-copyable and 1 on the rest gives the same share by construction (its gradient at initialisation equals that of the $f = 1$ run with the extra term to cosine 1.0000), and it does match $f = 0.5$: final best head 0.041, 0.047, 0.058 over three seeds against 0.056. The $f = 1$ run with the extra term does not: 0.069 and 0.081 over two seeds, as for $f = 0.3$ (0.078); identical configurations differ by 0.01–0.02. Onsets near the end of training are ill-conditioned (the head crosses 0.05 slowly), so we compare final heads. One candidate is that the oracle term keeps a CE-like gradient wherever the LM puts the target below ε/V, but $\varepsilon = 10^{-4}$, which shrinks that set, gives a stronger head (0.100), not a weaker one; the difference stays open.

When credit is removed matters. If formation were a pure credit clock, removing all copyable credit ($f = 1$) for a window of L steps before the onset would delay it by L wherever the window sits. It does not. Against the same-code-path baseline (mean onset 1,300), a window over steps 0–1,000 delays the onset by 785 steps, but one over 700–1,700 delays it by 1,696, more than the window itself. A second seed, predicted before the run to repeat the order, gives 587 and more than 1,620 steps against its baseline of 1,432 (the head reaches only 0.047 by the end), and halving or doubling its learning rate gives onsets of 1,268 and 1,327. During the later window the head stays flat (0.004) while the LM's loss on copyable positions rises from 3.71 to 3.97 nats: what the earlier credit had built is eroded by gradients from the other positions and must be rebuilt. Windows of 300 steps at 300, 600 and 900 cost 190–200 steps each, and one over 1,200–1,500, where the head is already 0.028, costs 294. The two blocks 0–300 and 900–1,200 together cost 155, not the sum of their separate costs (-176 and 198, the first within twice the run-to-run spread). Blocking after the onset (from step 2,000, head 0.145) does not freeze the circuit, as an error-driven rule with zero error would: within 200 steps the head falls to 0.078 and settles near 0.06, and the LM's loss on copyable positions jumps from 2.46 to 3.09 nats. Blocking from step 1,600 instead (head 0.117, learning rate still high) gives the same collapse: within 200 steps the gain from the repeated half falls from 0.74 to 0.29 nats, against 0.27 for a model that never received copyable credit, and stays there for 1,400 steps, so the residue is not an artefact of a small late learning rate. Half the credit is another matter: with $f = 0.5$ from step 1,600 the head holds (0.085, then 0.111 at the end), whereas $f = 0.5$ from the start reaches only 0.056. A second seed gives the same split (0.111 against 0.033). In a 305M model (onset near step 1,180), $f = 0.5$ from step 1,477 against from the start gives 0.158 against 0.047 and, at a second seed, 0.187 against 0.086 (we predicted a ratio of at least 1.5 before both). Keeping the circuit is cheaper than forming it, as for escape from a saddle. Upkeep is graded rather than switched: blocking $f = 0.75$ and 0.9 of the copyable credit from step 1,600 leaves heads of 0.087 and 0.053 and gains of 0.55 and 0.47 nats at the end (0.67 at $f = 0.5$, 0.33 at $f = 1$), whereas from the start neither forms a circuit at all. We had predicted a sharp upkeep threshold near $f = 0.8$; the head drops between 0.75 and 0.9, but the gain and the loss on copyable positions fall smoothly, so that prediction holds only for the head. We guessed that the collapse at $f = 1$ is active suppression by the positions where the head copies wrongly (the current token occurred before, its successor did not) and predicted that removing those positions from the loss during the block would freeze the circuit. It collapses further (head 0.018, gain 0.17 nats), whereas removing as many random positions does not (0.082, 0.34): those positions support the circuit rather than suppress it, and that guess is wrong. A ledger of the gradient explains the collapse but not that result. At checkpoints of a plain run we split the loss by class of position and measure each class's push along the induction head's own output weights ($-\langle g, w\rangle/\|w\|$). Copyable positions (16%) pay throughout ($+3.7$, $+8.4$, $+9.4$, $+9.4 \times 10^{-4}$ at steps 800, 1,200, 1,600, 2,400); once the head exists, positions whose current token occurred before without the bigram (38%) and positions whose target is new (23%) tax it (-3.2 and -4.6×10^{-4} at 1,600), together 86% of what copyable positions pay. Under sign descent (Adam's limit) the gradient without copyable positions moves 23% of the head's output-weight mass inward, and dropping the false-copy positions as well changes that little (-0.230 against -0.235), because new-target positions alone shrink it (-0.200). The instantaneous ledger also predicts shrinkage with half the credit (-0.093), where the head in fact dips (0.085 to 0.076) and recovers: upkeep comes from re-equilibration after the dip, which a single gradient does not show. We then guessed that the tax comes from heads that cannot abstain (queries and keys are normalised, so no position can absorb all attention) and gave every head a null slot, an unnormalised learned key with value zero. We predicted that blocking after the onset would then keep at least 70% of the head. It keeps 66% (0.120 to 0.080; 40–45% without the slot) and the loss on copyable positions rises as much as before (to 3.41 against 3.34). The ledger rules the mechanism out: with the slot the tax is still

84% of the pay, and the induction head puts only 8% of its attention on the slot at new-target positions (0–1% elsewhere); one last-layer head uses it heavily (37–45%). The slot may help in its own right (one run; the validation difference is about 2.5 times the spread of plain runs) (onset 1,094 against 1,223–1,404, final head 0.256 against 0.15–0.17, validation 4.251 against 4.262–4.278 over four plain runs), by a route our account does not identify. Last, the ledger's sign change (new-target positions push the head's output weights up at step 800, $+3.2 \times 10^{-4}$, and down from 1,200 on) suggested that removing those positions from the loss before the onset would delay it and removing them afterwards would strengthen the head. The second holds (final head 0.219 against 0.15–0.17, at a large cost to the full model, 4.525, since a quarter of the positions are no longer trained); the first is the wrong way round: removed for the first 1,000 steps, they bring the onset forward to step 834, about five times the run-to-run spread. Before the onset the output weights are not the bottleneck, so their ledger does not predict when the circuit forms. A ledger of the functional score (attention of the head to the successor of an earlier occurrence of the current token, on natural text, under a shared Adam-like normaliser) balances near zero while the score plateaus, but its sign predicted only one of three new removals before the onset (false-copy positions: onset 1,039 as predicted; positions whose target occurred earlier but whose current token is new: 946, predicted later; both false-copy and new-target positions: 1,519, predicted earlier). Removing any non-copyable positions early raises the copyable share and brings the onset forward (random 24% and 38%: 831 and 892), as the share law says; removing 60% starves the rest of the model and delays it.

G.9. Circuits compete: placement, blocking and release

Which head becomes the induction head is not predictable from the ledger either: two runs of one configuration (same seed, non-deterministic kernels) end with different induction heads (layer 7 head 2, layer 5 head 0), so the race is settled by noise; the per-head ledger ranked the eventual winner first of 64 at step 50 but 33rd, 9th, 48th, 11th and 12th at steps 100–600, and last at step 1,200 of the other run. Because noise settles the race, a small push should settle it instead. We added, for the first 300 steps only, an auxiliary loss pulling one chosen head toward induction attention (weight 0.05, a few per cent of the loss) and predicted that the head would win, keep winning after the push stops, and leave the final model unchanged; and that a head in layer 0, which has no earlier layer to compose with, could not hold the role. Layer 4 head 4 (never a winner unpushed) reaches 0.588 at step 300 and stays the induction head after the push ends (0.361 at 1,200, 0.317 at 2,400; next best 0.14); the onset moves from about 1,300 to 211. Layer 0 head 3 reaches 0.207 during the push and falls to 0.051 after it, while the circuit forms elsewhere (layer 7 head 0) on the usual clock (onset 1,410, validation 4.262). The final model is not unchanged, though: 4.231 against 4.263–4.278, though single pulled runs vary in validation (see below) and we do not read this as a gain. Where the circuit sits is a choice a few hundred steps of credit can make, and keeping it there costs nothing further, provided the place can compose. Kamin's design applied between heads gives blocking only in part. Pulling layer 5 head 6 alone for steps 300–600 leaves it at 0.317 at step 2,400; pulled for the same steps after layer 4 head 4 was pulled for 0–300, it ends at 0.254 (0.263 at a second seed), and pulled together with it for 0–300 at 0.240, while the first head is unaffected (0.320, 0.327 against 0.317 alone). We had predicted that the later head would collapse (below 0.15) and that two heads pulled together would exclude each other; neither happens, and two redundant induction heads coexist to the end (validation 4.203 and 4.209 against 4.231 and 4.232 with one; single runs, not read as a gain). The missing ingredient is a shared error. When instead the two heads' outputs are read out together through the LM head (a cross-entropy on the summed contributions, copyable positions only, weight 0.1, steps 300–800, no supervision of attention), the head formed first blocks the other completely, whichever it is: with layer 5 head 6 pulled into place for steps 0–300, layer 4 head 4 ends at 0.023 against 0.299 when the same compound (steps 300–800) is trained without pretraining, and with layer 4 head 4 pulled first, layer 5 head 6 ends at 0.027 (0.122 untrained, where the other head overshadows it); a second seed gives 0.020 against 0.311 and 0.060 against 0.265. On a second pair (layer 3 head 2, layer 6 head 1) the untrained compound forms both (0.312, 0.261), and pretraining either one blocks the other (0.037 and 0.070), a fall of 73–88%. We predicted the pretrained head to win and the other to stay below 0.15 in all eight runs (two pairs, two seeds), and it does. In a 305M model (16 layers, 16 heads; layer 6 head 3 and layer 11 head 5, the same schedule) the untrained compound forms both (0.183, 0.205) and pretraining either leaves the other at 0.058 and 0.045, below the half of the control we set as the criterion. Partial overshadowing without pretraining appears in the first pair at one seed only (Figure 4c). Blocking costs the model nothing here: on copyable positions the eight blocked runs reach 1.94–1.97 nats against 1.99–2.06 for the four untrained compounds (validation 4.19–4.23 against 4.22–4.27); the blocked runs also had the early pull, so we claim only that they are no worse, not better. (The second seed of the second pair is the one with an untrained compound of 0.096 and 0.267, and blocked heads of 0.035 and 0.028.) A later repeat of the second pair's untrained compound (seed 0, non-deterministic kernels) does not form the second head (0.060 at the end, layer 3 head 2 at 0.310), so without a head start that race too can be lost by noise; a repeat of the pulled run blocks it again

(0.021). How much head start does blocking need? Linear Rescorla–Wagner learning predicts a graded answer, the second head's final strength falling in proportion to the first's strength when they meet ($0.261(1 - V/0.56)$ for the second pair). Pulling the first head for 0, 50, 100, 150 and 300 steps (its score at step 300: 0.022, 0.053, 0.094, 0.221, 0.558) leaves the second at 0.261, 0.246, 0.023, 0.027 and 0.070: no gradation, but a switch between a head start of 0.053 and 0.094. Blocking between circuits is a race with a threshold, as the saddle-escape dynamics behind the hysteresis would have it, not a share of the remaining error; the linear prediction fails. We then asked whether the switch is set by the ratio of the two heads' strengths, as competitive growth with rates proportional to strength ($\dot{V}_i \propto V_i^2$, the bilinear circuit) would have it, and pulled both heads before they met. With scores 0.059 and 0.041 at step 300 (ratio 1.4) the second head is blocked (0.098 at the end), whereas from 0.053 and 0.027 (ratio 2.0) it formed; neither the ratio nor the first head's strength alone predicts these outcomes, so near the switch the race is settled by noise, as it is between unpushed heads. Far from it (0.209 against 0.035) blocking is certain (0.081). The fix does not undo this. Bush–Mosteller credit at the circuit level means giving each head its own error as well, a readout of its output alone added to the shared one in the compound phase (weight 1 each); we predicted that the blocked head would then form (above 0.2). It does not: 0.019 in both pairs, against 0.070 and 0.027 with the shared error alone. Its own error gives the late head credit, but credit alone does not lift a circuit off its saddle within 500 steps (a head trained by its own readout alone for 500 steps did not become an induction head either), while the formed head keeps growing on the same credit. This bounds the remedy of the main text: it restores what the LM head learns, and the circuit only half-way (Appendix G.7); it does not reverse a race already lost. What does reverse it is removing the winner for a while. Zeroing layer 4 head 4 only for steps 1,200–1,800 lets other induction heads form in that window (layer 4 heads 1 and 7: 0.288 and 0.255 at 1,800), and after it is restored all of them stay (0.307, 0.283 and 0.250 at the end; the others' total 1.42, as in unpushed runs). Upkeep being cheap predicted this coexistence; competitive exclusion under the shared error predicted that one would be pushed out again, and none is. Competition for one error acts on formation, not on circuits that already exist. A head committed to another role first stays out of the race, the analogue of latent inhibition: pulling layer 3 head 2 to attend to the previous position for steps 0–300 (attention there 0.556 at step 300 against 0.010 unpulled, still 0.191 at the end) and then running the same shared-readout race leaves it at 0.024 against 0.312 in the untouched race, while layer 6 head 1 wins (0.272 against 0.261). We predicted below 0.2, and it is, but a control shows that this is not inhibition: pulled instead toward a pattern no circuit uses (the position five back; attention there 0.484 at step 300), the head drops the pattern (0.008 at the end) and competes as if untouched (0.297 against 0.312; the other head 0.236). The previous-token pattern is kept (0.191) because the circuit needs it, and a head holding that role is not free to become the induction head; latent inhibition, in the sense of a pre-exposed cue learning less, fails. A repeat at a second seed (seed 2) cannot separate the readings: there the untouched race is itself won by layer 6 head 1 (layer 3 head 2 at 0.061), and both pre-exposed runs leave layer 3 head 2 as low (0.018 previous-token, 0.031 useless pattern), so the test discriminates at seed 0 only. Across seeds, layer 6 head 1 forms in three of the four untouched races of this pair (0.261, 0.267, 0.278; 0.060 in one repeat). In both runs the winning head grows faster than in the untouched race (0.394 and 0.318 at step 800 against 0.151). We guessed it reads the pre-exposed head as its previous-token stage and predicted that zeroing that head would cut its score by a third; it does not change it (0.394; 0.282 against 0.272 at the end), although zeroing the plain model's main previous-token head (layer 1 head 4) does lower its induction head (0.228 to 0.198), so the faster growth has another cause. The cause is the pull's gradient reaching the shared residual stream: repeating the useless-pattern run with the pulled head's query and key computed from a detached residual (it still learns the pattern, 0.473) removes the faster growth (0.148 at step 800 against 0.151 untouched and 0.318 with the gradient flowing), as we predicted. We also guessed, after seeing the two runs' validation (4.223 flowing, 4.269 detached), that the validation gains of pulled runs come from the same spill-over, and tested it on another head and pattern without the readout: the prediction is reversed (4.275 flowing, 4.246 detached), so single runs' validation differences of this size are not interpretable and we draw no conclusion from them. The model's own loss is such a shared error, and it blocks too: unpushed runs grow two or three induction heads (best, second and third 0.228/0.218/0.147 and 0.263/0.237/0.208), but once a head has been pulled into place the best of the others reaches only 0.142 (pulled layer 4 head 4) and 0.078 (layer 5 head 6, pulled for steps 300–600), the second best 0.065–0.066; when the pull fails to place a circuit (layer 0) the others form as usual (0.267/0.223/0.137). With a second seed, at step 1,200 (before any removal) the best other head is 0.063. In a 305M model this blocking through the model's own loss fails: with layer 6 head 3 pulled into place (0.37 at step 1,200) two heads of the same layer reach 0.269 and 0.235 (others' total 1.55), so there was nothing for its removal to release (the best other rose to 0.331). We had predicted a release of at least a factor of two; with many more heads (256) several induction heads coexist, and only the explicit shared readout above blocks at that size. Supervising two heads' attention directly bypasses this, which is why both survived above. Removing the blocker releases the block, as the shared-error rule predicts: with layer 4 head 4 placed and its output zeroed from step 1,200, the best other head (layer 4 head 5) goes from 0.079 to 0.165, 0.293 and 0.336 at steps 1,350, 1,500 and 1,800 and the other heads' total returns to that of unpushed runs (1.44 against 1.42–1.50). We predicted release (best other above 0.2) and a re-formation

faster than the first (within 600 steps); it takes about 300, against about 1,300 for the first formation, in the same layer as the removed head, so the prerequisites it built are reused. A second seed repeats the release more slowly: the best other head goes from 0.063 at step 1,200 to 0.145, 0.202 and 0.276 at 1,500, 1,800 and the end (others' total 1.38). Validation is 4.237 (4.231 without the removal). A first attempt that pretrained one head by the readout alone failed our validity check (the head had not become an induction head after 500 steps) and is not counted. *Unblocking by a stronger or weaker outcome.* In animals a blocked cue learns when the outcome is made larger than the first cue predicts (upshift; Kamin, 1969); linear Rescorla–Wagner predicts this, and only the attentional theory of Pearce and Hall (1980) predicts that a weaker outcome (downshift) also unblocks. We changed the outcome of the shared readout in the compound phase alone, at a fixed rate: the readout loss is $T \cdot \mathrm{CE}(z/T)$, whose gradient $p_T - y$ is a larger error than before for $T = 3$ (the summed heads now fall short) and a smaller one for $T = 1/3$, against a rate-only control that triples the readout weight. We predicted that upshift would unblock the second head (above 0.10 at step 800) and downshift and rate would not. Neither shift unblocks, in either pair: the second head's induction score at step 800 is 0.019 (upshift), 0.035 (downshift) and 0.013 (rate) against 0.009 for the repeated blocked run in the second pair, and 0.028 and 0.024 for up- and downshift in the first (0.020–0.041 at the end throughout). The upshift does reach the heads' outputs: the direct logit attribution of the two heads to the target on copyable positions rises from 46 and 23 (blocked run) to 113 and 34, but the increment goes mostly to the head that has already won (Proposition 6). The Pearce–Hall prediction fails, and so does our linear-rule prediction for upshift. A second seed of the second pair agrees: 0.095 (upshift) and 0.057 (downshift) at step 800, 0.087 and 0.044 at the end, against 0.028 blocked without a shift; the upshift value is the closest any run came to our 0.10 criterion. (One run per condition and seed.) (Figure 4 in Section 10.)

G.10. Why the race has a threshold: circuit depth

Linear Rescorla–Wagner shares the outcome in proportion to what each cue has already learned plus an equal increment; between heads we instead see a race with a threshold. A circuit's strength, however, is not one weight but a product of several. Deep linear networks learn each mode on a sigmoidal schedule whose speed grows with depth and with the mode's current strength (Saxe et al., 2014; Arora et al., 2018), and small initialisation of deep parametrisations gives the "rich" regime (Woodworth et al., 2020); we apply the same calculation to two circuits that share one error, where it decides which of them learns.

Proposition 6 (Depth sets the race)**.** *Let two circuits have strengths $S_i = \prod_{k=1}^{p} u_{ik}$ with balanced factors ($u_{ik} = S_i^{1/p}$), trained by gradient flow on a squared shared error $\frac{1}{2}(\lambda - S_A - S_B)^2$ with rate η. Then $\dot{S}_i = p\,\eta\,S_i^{2-2/p}\,(\lambda - S_A - S_B)$, and the log-ratio obeys $\frac{d}{dt}\log(S_B/S_A) = p\,\eta\,(\lambda - S_A - S_B)\,(S_B^{1-2/p} - S_A^{1-2/p})$. For $p = 1$ the weaker circuit catches up (the linear rule), for $p = 2$ the ratio is conserved (the head start is kept in proportion) and for $p \geq 3$ the stronger circuit pulls away; the race stops when the error is gone, so the later circuit keeps an amplified deficit, and its final share goes to zero for any head-start ratio above one as the initial strengths go to zero (winner-take-all from small initialisation); near a ratio of one, noise decides.*

Proof. Each factor obeys $\dot{u}_{ik} = \eta\,(\lambda - S_A - S_B)\,S_i/u_{ik}$; balance is conserved by this flow, so $\dot{u}_i = \eta\,e\,u_i^{p-1}$ and $\dot{S}_i = p\,u_i^{p-1}\dot{u}_i = p\eta e\,S_i^{2-2/p}$. Dividing gives the log-ratio. □

An induction head is at least a three-factor circuit (a previous-token head's output, the induction head's query–key match, its output–value map), so the proposition predicts the race we observe; a head whose keys are taken from the previous token by construction (one composition fewer) should share more gracefully. It also predicts that a larger shared error does not unblock: it multiplies both rates by the same factor, which speeds up the leader's escape, as the upshift test above finds (we wrote the proposition after the first upshift run and before the other five). Read literally, with induction score as strength, it would block at the head-start ratio of 2.0 where the second head formed; scores near the untrained floor (0.02–0.03) are not strengths, so we use only its qualitative claims. We tested one prediction of the depth reading before the run: in a model whose second half of heads take their keys from the previous token (one-layer induction), we predicted graded sharing between two such heads (layer 3 head 6, layer 6 head 5) under the same shared readout. It fails. Without a head start one head already wins (0.636 against 0.010 at step 800), and with head starts of 50, 100 and 300 steps the second stays at 0.011–0.012 (0.017–0.019 at the end), so the shallower circuit races as sharply. Its query–key and output–value maps are themselves products of two matrices each, so a "one-layer" induction head is not a two-factor circuit and the test did not isolate depth. (One run each.)

Depth in isolation. To vary only the number of factors, we pulled two heads into induction together (layer 4 head 4 and layer 5 head 6, steps 0–300), so that both form, and from step 300 read their detached outputs through gates, $\mathrm{CE}(W(g_A o_A +$

$g_B o_B)$) with the LM head W fixed, so that only the gates learn and the model is untouched. Each gate is a product of p scalars, and fifteen independent gate pairs share one run: $p = 1, 2, 3$ under SGD (rate 0.003) with head starts $r = g_A/g_B = 1, 1.5, 2, 4$ at $g_B = 0.02$, and $p = 3$ under Adam (rate 0.003, $r = 1, 2, 4$). Features, loss and the function class are identical across pairs; only the parametrisation differs. We predicted that the head start would be erased for $p = 1$, kept for $p = 2$, amplified for $p = 3$, and erased under Adam, which moves every factor at the same speed. We measure the amplification $\log(\text{odds}_r/\text{odds}_1)/\log r$ of the gate odds g_A/g_B (0: erased, 1: kept, above 1: amplified):

steps after onset of the readout	$p = 1$	$p = 2$	$p = 3$	$p = 3$, Adam
100	0.09–0.17	0.95–0.98	1.10–1.12	0.40–0.44
300	0.05–0.08	0.74–0.80	1.16–1.42	0.02–0.03
700	0.01	0.38	0.79–1.04	0.00
end (2,750)	0.00	0.00	0.00	0.00

All four predictions hold in the early phase (ranges over r). A quantitative criterion we set, that at step 1,000 the second gate's share at $r = 2$ would be below half of its share at $r = 1$, fails (0.257 against 0.201). Because the two heads' outputs are not identical, the loss has one optimum, and every pair reaches it by the end (gates 0.15 and 0.23): with distinct features a head start is transient however deep the parametrisation. Permanent blocking needs what the circuits add, a saddle that the late head must still escape when the error is gone. The Adam column also leaves a question open: the circuits themselves are trained with Adam and still race, so per-coordinate normalisation of matrix factors does not equalise their growth as it does for scalar ones. (The pairs share each run, so they differ only in their gates. A second seed gives 0.05–0.09, 0.81–0.84, 1.21–1.40 and 0.03–0.04 at 300 steps, the same ordering, and the same transience.) With distinct features the head start is transient; the proposition is about identical cues, whose optimum is a ridge ($g_A + g_B$ fixed) rather than a point. We therefore repeated the run with both gates of every pair reading the same head (layer 4 head 4), predicting before the run that the second gate's final share would move toward one half for $p = 1$ and under Adam, stay at $1/(1 + r)$ for $p = 2$, and fall below it for $p = 3$. All four hold. For $r = 1.5, 2, 4$ the share starts at 0.400, 0.333 and 0.200 and ends at 0.485, 0.470, 0.410 ($p = 1$); 0.400, 0.333, 0.200 ($p = 2$, unchanged to three decimals over 2,750 steps); 0.322, 0.237, 0.128 ($p = 3$, fixed from about step 1,000, once the error is spent); and 0.406, 0.284 under Adam ($r = 2, 4$). With identical cues, a three-factor parametrisation turns a 2:1 head start into 3.2:1 for good, a one-factor one into 1.1:1, and the same parameters under Adam into 1.5:1. A second seed gives 0.466, 0.333, 0.244 and 0.401 for $r = 2$ (against 0.470, 0.333, 0.237 and 0.406), with $p = 2$ again unchanged to three decimals for every r. Why, then, do circuits trained with Adam still race? We guessed that Adam's normaliser is set by a parameter's other signals, so that a parameter serving other objectives moves like an SGD parameter on the shared error, and tested it by adding to every gate factor a random-sign gradient as large as the readout loss itself (a proxy for those objectives), predicting amplification for $p = 3$ under Adam. It fails: under Adam the noise sets each step's direction, and the shares wander (0.16–0.65 at the end, with no order by p or r), while under SGD the same noise leaves the results above unchanged ($p = 2$: 0.327 and 0.207; $p = 3$: 0.252 and 0.145 for $r = 2, 4$). The race in Adam-trained circuits remains unexplained by this account. (One run.) A random signal is the wrong proxy: a head's other objectives pull it consistently. We therefore added instead a constant pull c toward zero on every gate factor (identical cues, head start 2:1, start share 0.333), predicting that under Adam the later gate, whose shared-error gradient falls below c first, would flip sign and be pushed out (share below 0.15 for $p = 2, 3$ at some c, against at least 0.38 at $c = 0$), that $p = 1$ would not race (both gates get the same gradient), and that Adam would race harder than SGD. At the end of training (two seeds, agreeing to 0.005):

later gate's share	$c = 0$	$c = 0.01$	$c = 0.03$
Adam, $p = 1$	0.467	0.467	0.467
Adam, $p = 2$	0.427	0.392	0.267
Adam, $p = 3$	0.402	0.285	**0.000**
SGD, $p = 2$	0.333	0.330	0.322

A consistent competing pull turns Adam's erasure into a race, ending in complete exclusion of the later gate at $p = 3$ and $c = 0.03$; $p = 1$ does not race; and Adam races harder than SGD at the same pull. The $p = 2$ criterion is missed (0.267). At $c = 0.1$ the pull drives gates through zero and the readout is no longer interpretable. This would explain Adam-trained heads racing if heads behaved like these gates, each also serving other objectives; the tests on the heads below say they do not. We then tried it on the heads: in both pairs, with the readout extended to steps 300–2,000, we

removed the LM loss's gradient from the late head (its output reaches the residual stream without gradient), predicting that without competing objectives it would no longer be excluded (above 0.15 at step 2,000, against below 0.10 in the control). It fails: 0.015 and 0.013 against 0.010 and 0.014. The test does not isolate the mechanism, because the LM loss is also the late head's main source of induction credit (a head trained by its own readout alone does not form either; Appendix G.9), so we repeated it keeping the LM loss's gradient into the late head only at induction-copyable positions, its induction credit, and removing it everywhere else, its competing objectives. We predicted formation (above 0.15); it stays blocked (0.051 and 0.003). Competing objectives under Adam explain the gates' race but not the heads'. A last candidate is that the late head's remaining signal falls below the gradient noise; halving the noise (64 sequences per step, 200M tokens, the same 3,051 steps and schedule) should then free it (above 0.10, predicted before the run). It does not: 0.031 at the end, while at the same batch without a head start the same head forms (0.263). Last, attention weights are bounded and a saturated head explains its targets alone, so we bounded the gates ($g = M\sigma(\prod_k u_k - 5)$, $M = 0.35, 0.5$), predicting a race under Adam (later gate's share below 0.2). Under SGD the bound does create one, even for one factor (0.274, and 0.132 for two; a saturating nonlinearity acts like an extra factor), but under Adam it does not (0.32–0.43). AdamW's weight decay is a consistent pull toward zero on every parameter, like the pull that made Adam's gates race, so we removed it ($wd = 0$), predicting that the late head would escape (above 0.10); it does not (0.056; the same head grows in the unpushed control, 0.104 at step 800). Layer order is not the cause either: in both pairs the head formed first blocks the other whether it sits above or below it. Why Adam-trained heads race remains open. (One run per condition.)

G.11. A by-product: withholding frequent targets early

Removing the positions whose target occurred earlier but whose current token is new, for the first 1,000 of 3,050 steps, improves the final model: validation 4.222, 4.224 (repeat) and 4.230 (second seed) against 4.264, 4.268, 4.278 and 4.263 (second seed) for plain training and 4.256 for the same fraction removed at random. The model is worse at step 1,000 (4.97–5.00 against 4.93–4.94) and better from step 2,000 on, on copyable and other positions alike. We guessed a context-unigram shortcut that blocks the induction circuit; a probe of it (the boost to tokens present in an unrelated prefix) is unchanged (0.061 and 0.073 nats at steps 800 and 1,200 against 0.058 and 0.067). These targets are mostly frequent tokens (median frequency rank 6; 83% in the top 100), and removing every position whose target is one of the 10 most frequent tokens (24% of positions, regardless of context) gives most of the gain (4.238), consistent with the dominance of frequent classes in early gradients (Kunstner et al., 2024); the context condition adds 0.008–0.016, within twice the spread. Half the default learning rate gives 4.233 by itself, so part of the gain may come from smaller effective early updates rather than from which positions are removed; we have not tested whether it survives longer training or a tuned rate. A removed credit is not a paused credit; the circuit has to be paid for continuously, and formation costs more than upkeep. A logit lens locates the lost stage in depth. On the repeated half, the CPLM backbone finds the target abruptly at layer 19 of 30 (135M; layer 22 of 32 at 360M), where its median rank among real tokens falls from 13,294 to 781 (10,082 to 1,225) and the lens probability of <copy> jumps (0.59 at 135M); the target then reaches rank ≈350 at the output but is never sharpened, while the last layers put all mass on <copy> (1.00 on the repeated half, 0.73–0.95 on the first). The vanilla models give the same target -0.4 nats at the output. The backbone computes which token to copy and then writes "copy" instead of the token. We then predicted that repairing only the layers after the match would restore the LM head's copying quickly, since the match is already computed. It does not: fine-tuning CPLM-135M with the plain LM-alone loss for 800 steps of 4×1024 tokens raises the copying gain from 3.17 to 3.09 (last 2 layers), 3.29 (first 10), 3.71 (last 10) and 4.59 (all 30 layers), against 10.77 for the vanilla model. Later layers matter more, but the writing is spread over depth, as the vanilla lens rising from layer 10 to the output suggests, and it is relearned slowly. We predicted that the writing would also be erased progressively over the 1.7B run; it is not. Over the released checkpoints at 42B, 84B, 126B, 168B and 200B tokens the LM-head copying gain is 3.52, 2.63, 3.05, 4.45 and 4.73 nats, while the best induction head stays at 0.91–0.95 and the number of induction heads at 2. The late rise may come from the learning-rate decay, but the vanilla full-attention baseline has no intermediate checkpoints to separate it. It is also what the late escape of Proposition 3(b) predicts for an Adam-trained model whose blocked and unblocked tokens share parameters, and it matches the slow decay of the released run's delegation (Section 4); we cannot tell the two accounts apart with the released checkpoints. We also predicted that the released instruction-tuned checkpoint would show further erasure. It does not: after SFT the CPLM-1.7B LM head's copying gain rises from 4.73 to 5.42 nats (induction heads above 0.3: 2 to 4), and the vanilla model's from 10.26 to 10.48. The model card gives the data (Tulu-3, up to 16k tokens, 12,138 steps) and says the full-attention models were extended with YaRN ($\times4$) during SFT, which changes the positional encoding our repeated-sequence probe relies on, but not whether the copy head stayed in the loss. Comparing the weights answers that: the copy head's projections moved by 12% (relative norm of the change) against 6% for the backbone's layers, so it was trained during SFT. The prediction therefore failed in the setting it was

made for, unless the YaRN change of positions accounts for it. The sliding-window variant was instruction-tuned without YaRN, and there the prediction holds: on a sequence repeated at distance 128 (inside the window) the CPLM-1.7B LM head's copying gain falls from 3.45 to 2.79 nats after SFT while the vanilla model's rises from 9.46 to 10.10 (its copy head also moved during SFT: 11% against 6% for the backbone).

G.12. A training-only scaffold

77M, 100M tokens, LM alone on validation (no helper at test time); one seed each, three plain runs for the spread.

	val	copyable	other	induction head
plain (three runs)	4.262 / 4.275 / 4.278	2.09–2.15	4.71–4.72	0.156
pointer helper throughout	4.347	2.87	4.66	0.181
pointer helper throughout, extra term	4.198	2.12	4.63	0.147
pointer loss, first 20% (no mixture)	4.214	1.96	4.689	0.218
pointer loss weighted by detached $c/(c+q)$, first 20%	4.223	2.00	4.688	0.221
pointer helper, first 20% (scaffold)	**4.172**	1.99	**4.629**	0.226
scaffold with the extra term (LM not relieved)	4.180	1.97	4.636	0.222
scaffold, gate input detached from the trunk	4.198	2.00	4.660	0.190

The auxiliary-loss baseline (Deshpande and Narasimhan, 2020) trains the same head equally well but gains less, and the difference is on positions the helper does not cover. We tested two explanations and both failed: that blocking relieves the LM during the scaffold (adding the extra term, so that the LM is not relieved, leaves the gain: 4.180), and that the mixture's responsibility weighting of the pointer is what matters (weighting the auxiliary loss by the detached responsibility gives 4.223). The two mixture runs (4.172, 4.180) and the two auxiliary runs (4.214, 4.223) differ by more than the plain runs among themselves (0.016), but why the mixture helps more remains open: cutting the gate's gradient into the trunk, a signal of when copying is reliable that the auxiliary loss lacks, gives 4.198, halfway between the two groups and within the spread of single runs. Related interventions accelerate induction heads by reweighting data (Wang and Murfet, 2026) or slow early upper-layer attention (Zhu et al., 2026); at 300M tokens the same scaffold (helper for the first 20%, i.e. three times as many steps) ends at 3.873 against 3.852 for the plain model; its gap falls from $+0.094$ at step 2,000 to $+0.022$ at the end but does not close, so the 100M-token gain does not survive longer training. We did not separate the length of the blocked period from training length.

G.13. Delegating after the onset

The hysteresis of Appendix G.4 (forming the circuit needs more credit than keeping it) predicts that the order of training decides whether a copy-head LM keeps its own circuit. Before a step t_d we train the copy head and gate on the mixture with the LM's probability detached and give the LM plain cross-entropy (no delegation); from t_d on, ordinary mixture training. 77M, 100M tokens, one seed; ordinary training gives 3.994 here and 4.000–4.003 over three seeds in earlier runs; repeats of one configuration of this model spread by about 0.0035.

delegation from step	head at t_d	head at end	val (full model)
0 (ordinary training)	–	0.006	3.994
800	0.013	0.020	3.995
1,200	0.089	0.075	**3.981**
1,600	0.147	0.115	3.985
never	–	0.208	4.026
1,200, repeat	–	–	3.985
extra term, weight 0.5 (eight runs)	–	–	3.975–3.985
1,200 and the extra term (0.5), two runs	0.122	0.204	3.996 / 3.993

We predicted before the last two runs that delegating below the onset would leave no circuit and delegating above it would keep one; both held. Delegating at step 800, where this run's head was still 0.013, gives the same model as delegating from the start; at 1,200 (head 0.089) the head drops to 0.054 and then holds and grows to 0.075, and the full model gains 0.009–0.013 over two runs, about three times the run-to-run spread, with no extra loss term. Later delegation keeps more

circuit but less of delegation's benefit; never delegating keeps the most circuit and is worst. The extra term gives the same: eight runs of it, two seeds and repeats under functionally identical code, range from 3.975 to 3.985 (mean 3.980, standard deviation 0.0035), a larger run-to-run spread than the 0.002 of three seeds of the plain copy-head model, so order and the extra term cannot be told apart. Combined, the two do not add: delegating at 1,200 with the extra term keeps the strongest circuit (0.204) but ends at 3.996 and 3.993, no better than ordinary training. Logged gates rule out the explanation we expected (that the gate trusts the helper less): from step 1,500 to 2,000 it is 0.14 with the extra term alone and combined. What differs is complementarity: combined, the LM alone is the best of the three (4.144 against 4.152 and 4.228 on a fixed batch), but it already copies (target probability 0.279 where the target is in the context), so the helper adds the least (0.227 nats against 0.252 and 0.315). A rule that needs no step count, delegating once the LM's best induction head exceeds 0.05 (the onset definition, checked every 50 steps), switches at step 950 and ends at 3.985, as we predicted (3.981–3.985). We meant to show that a fixed step does not transfer, using sequences of 512 tokens, where we expected a later onset; the onset came at about the same step (head 0.036 at 1,200), delegation at 1,200 kept the circuit (0.058 at the end; 4.080) as did the rule (switch at 1,350; 0.064; 4.082), against 0.015 and 4.096 for ordinary training, so that test did not separate the two. Since delegation at head 0.036 keeps the circuit and at 0.013 does not, the upkeep threshold lies between them. In a 305M copy-head model (onset near step 1,180), delegating from step 1,200 ends at 3.824 against 3.838 for delegation from the start, a gain of 0.014 (four times the spread of the 77M model), with the LM's best induction head at 0.141 against 0.028. We had predicted a gain below 0.005, since blocking by the copy head is weaker at this size; the gain is larger than predicted. A second seed, where we predicted a gain above 0.007 (twice the spread), gives 3.836 against 3.844 (0.008) and heads of 0.242 against 0.033.

G.14. Shapley credit and blocking between helpers

For n components the Shapley value of component i is $\sum_{S \not\ni i} \frac{|S|!(n-|S|-1)!}{n!}[v(S \cup i) - v(S)]$ with $v(S) = \log p_S(y)$, p_S the mixture restricted to S with its gate renormalised; giving every component its Shapley gradient is minimising $-\sum_S w_S \log p_S$ with $w_S = (|S|-1)!\,(n-|S|)!/n!$, since $\log p_S$ reaches only the members of S. For $n = 2$ this is $-\frac{1}{2}(\log p + \log q + \log c)$: the extra term, plus the helper's own term. Plain joint training is the other classical rule: since $v(N \setminus i)$ does not depend on component i's parameters, $\nabla_i v(N) = \nabla_i[v(N) - v(N \setminus i)]$, the gradient of its Clarke-pivot (VCG) payment (Vickrey, 1961; Clarke, 1971; Groves, 1973). A mechanism that pays each player its marginal contribution to the grand coalition pays nothing to a player made redundant by another, which in learning means that the redundant player stops learning; Shapley pays the average marginal contribution over orders of arrival, including the order in which the player is alone. We test three components on the key-to-label task of Appendix C (SGD, 20k steps): the LM and a second MLP B, both trained, and a frozen helper A perfect on half the keys, with a softmax gate that starts trusting A. Accuracy of each deployed coalition (covered half / other half):

	LM	B	LM+B	LM+B + A
plain $-\log p$	0.02 / 0.39	0.02 / 0.40	0.03 / 0.76	1.00 / 0.76
extra term on the LM only	1.00 / 1.00	0.01 / 0.07	1.00 / 1.00	1.00 / 1.00
own term on every trained component	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00
Shapley loss	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00

Without any term both trained components are blocked by A and split the rest between them (0.39 and 0.40), so neither works alone; as VCG credit predicts, they avoid redundancy: both are right on $0.39 + 0.40 - 0.76 = 0.03$ of those keys, against 0.16 if they were independent; the same holds in released Copy-Paste LMs: on held-out tokens whose target is in the context, CPLM's LM head is right (top-1) on 71.8% / 75.3% (135M / 360M) of the tokens the copy head gets right, against 83.9% / 84.4% for the vanilla model, and on 42.7% / 43.7% of those the copy head gets wrong, against 41.9% / 43.0%: the LM head's whole loss is where the copy head is right, so the two overlap less than a model trained without the copy head does; repairing only the LM makes it overshadow B everywhere. Blocking is a property of every pair of components that share a likelihood, not of the LM. In the 77M copy-augmented LM (100M tokens) the same overshadowing appears: with the extra term on the LM only, the copy head alone has 3.187 nats on tokens whose target is in the context; adding its own term (the two-component Shapley loss) brings it to 2.525. It does not pay for the deployed model, though: the full mixture goes from 4.002 to 4.078 and the LM alone from 4.198 to 4.299, so with a shared trunk fair credit makes the components compete for representation, and the loss to use depends on which coalitions will be deployed (one seed each). Between two helpers in a language model we did not find it. A 77M LM (sequence 512, 100M tokens) trained with the copy head and a frozen 30M-token LM as a second helper, all three in one likelihood, has a copy head alone of 3.451 nats

on seen targets against 2.987 without the expert; but the LM itself is locked in there (10.64 alone) and the shared trunk with it. With the extra term on the LM, so that the trunk trains normally, the copy head alone is 3.051 against 3.030 without the expert: the frozen expert and the copy head explain different tokens and the second helper costs the first almost nothing (one run each). VCG credit predicts that blocking between helpers needs overlap: a frozen expert that does copy (the plain 77M model, which has induction heads) should cost the copy head more. It does, slightly: 3.097 against 3.051 for the non-copying expert and 3.030 for none; the order is the predicted one, but the effect is small against run-to-run variation of a few hundredths. Jointly trained ensembles are known to degrade: members collude to inflate an apparent diversity that does not generalise, and interpolating between independent and joint training helps (Jeffares et al., 2023). The effect here is different in kind: blocking and niche partition lower each member's own accuracy on the training data itself, and Shapley credit fixes the interpolation weights rather than leaving them as a hyperparameter.

G.15. Overexpectation and superconditioning

Rescorla–Wagner also predicts overexpectation: two cues trained separately to the outcome and then presented together over-predict it, and the shared error weakens both; compound reinforcement adds nothing to such cues (Rescorla, 1970) and removes strength from cues conditioned to asymptote (Kremer, 1978). We train two MLPs separately to convergence on a key-to-label task with 30% label noise (the best probability of the true label is 0.72), then train them jointly for as long again (one run each).

each member alone, probability of the true label	A	B
after training alone	0.712	0.707
joint, product (logit sum)	0.303	0.289
joint, product + each member's own term	0.506	0.505
joint, mixture (fixed $\frac{1}{2}$)	0.995	0.429

A product over-predicts and both members become under-confident (alone 1.38 $\rightarrow$ 1.74 and 1.77 nats), as overexpectation predicts; their own terms halve the damage but do not remove it. We expected a mixture of two calibrated members to leave them unchanged; instead they partition the label distribution, one member taking the clean label with near certainty and the other the noise, so that neither is calibrated alone (1.38 $\rightarrow$ 2.35 and 1.55): niche partition again.

The mirror image of blocking is superconditioning: a cue paired with an inhibitor, which predicts that the outcome will not come, is conditioned more strongly than alone. In a product this has an exact form. A frozen helper that puts $-m$ on the clean label of half the keys (same noisy task) forces the LM to add m to its own margin there: the LM alone's true-label log-odds margin exceeds that on the uncovered keys by 1.05, 2.04 and 3.05 for $m = 1, 2, 3$, and its probability of the true label rises from the calibrated 0.71 to 0.88, 0.95 and 0.98. With its own term at equal weight the excess is 1.27 for $m = 2$, near the $m/2$ that balancing the two losses gives. A model trained in a product with a frozen biased model, as in debiasing by products of experts (Clark et al., 2019), is in this regime wherever the frozen model is wrong.

H. Additional copy-head results

H.1. Compute-optimal training

At 100M training tokens, the plain Transformer needs about 2.1×, 1.7× and 1.6× the tokens (77M, 163M, 305M) to match the copy-augmented model trained with the extra term (Table 1).

Compute-optimal training. At about 20 tokens per parameter (77M, 1.5B tokens of a separate FineWeb-edu shard; first seed below, second at the end of this paragraph) the copy-augmented model trained with the extra term reaches 3.533 against 3.591 for the plain Transformer. The gap narrows during training (-0.42 at 1k steps, -0.12 at 9k, -0.063 at 37k of 45.8k) but does not close. Fitting $L = E + A\,D^{-\alpha}$ to the plain model's 100M, 300M and 1.5B-token losses (three points, no error estimate, and the first two on a different shard) puts the plain model at about 3.1B tokens for 3.533, roughly 2×. Its LM head alone is 3.620, 0.03 worse than the plain Transformer. Trained with the original mixture loss the same model reaches 3.528 but its LM head alone is 3.774, 0.18 worse than the plain Transformer: at this length delegation is larger than at 100M tokens ($+0.12$), and the extra term reduces it to 0.03 at a cost of 0.005 nats of full-model loss. Blocking the mixture's gradient into the LM head entirely (it learns from its own cross-entropy only; the mixture trains the copy head and gate) gives 3.545 with an LM head of 3.611: still 0.02 behind the plain Transformer. The remaining 0.02–0.03 is therefore

not delegation through the mixture gradient, which this variant removes by construction, but the shared backbone's capacity spent on features for the copy head, and the extra term is the better trade-off of the two. A second seed of the three main runs, with predictions fixed before it (full model at least 0.04 better than plain; copy head with and without the term within 0.015; LM head alone more than 0.10 behind plain without the term and less than 0.06 with it), gives 3.534, 3.528 and 3.594 for the full models and, on one shared validation sample, LM heads 0.166 and 0.022 behind the plain model (0.168 and 0.024 for the first seed on the same sample): all four hold.

H.2. Training trajectory of the released 1.7B model

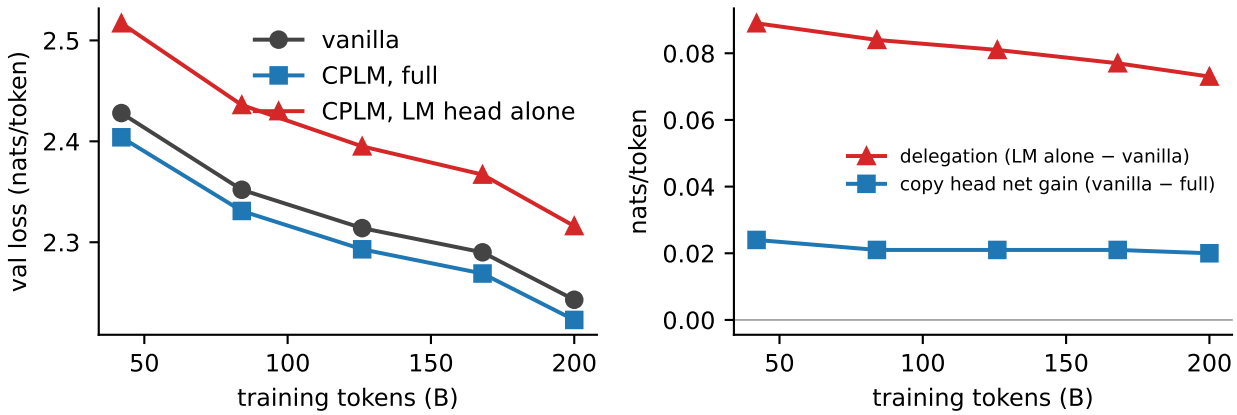


Figure 5. Released CPLM-1.7B and its vanilla twin over training (sliding-window attention). Left: validation loss. Right: the LM head's delegation is 4× the copy head's net gain at every checkpoint.

H.3. Fix at every scale

model	tokens	full model		LM head alone	
		copy head	+ plain CE	copy head	+ plain CE
small	30M	4.640 / 4.650	4.643 / 4.656	5.068 / 5.077	4.981 / 4.999
small	120M	4.298 / 4.305	4.290 / 4.287	4.658 / 4.668	4.507 / 4.515
77M	100M	4.002 ± .002	3.979 / 3.989	4.381	4.185 / 4.200
77M	300M	3.751 / 3.755	3.741 / 3.737	4.058 / 4.068	3.870 / 3.866
163M	100M	3.906 / 3.908	3.914 / 3.925	4.277	4.110 / 4.134
305M	100M	3.838 / 3.842	3.840 / 3.851	4.190 / 4.194	4.021 / 4.038
305M	300M	3.557 / 3.557	3.528 / 3.530	3.836 / 3.829	3.630 / 3.632
77M, CPLM-style head	100M	4.064 / 4.071	4.031 / 4.036	4.472 / 4.482	4.261 / 4.282

Table 11. Two seeds each. Vanilla LM: 5.137, 4.472, 4.261, 3.857, 4.116, 4.023, 4.261 respectively (163M copy-head LM-alone from one saved seed). The last row reproduces the CPLM head (a `<copy>` slot competing with the vocabulary as the gate, and a learnable sink key): it delegates more (+0.21) and the fix improves both its full-model loss and its LM head. The extra term costs nothing at train time (the fused LM cross-entropy is computed anyway).

H.4. A 608M model

To test the delegation and the fix above the largest size of Table 11, we trained three 608M models ($d = 1536$, 16 layers, 100M tokens, learning rate 1.5×10^{-3} fixed in advance, two seeds each) and evaluated all three on one validation sample (131k tokens). We predicted before the runs that the copy-head model's LM head alone would be more than 0.10 nats behind the vanilla model, that the extra term would bring it within 0.05, and that both copy-augmented models would beat the vanilla one by more than 0.1 nats. All three hold at both seeds.

608M, 100M tokens	full model	LM head alone	seen	new
vanilla	3.923 / 3.928	3.923 / 3.928	2.535 / 2.541	5.540 / 5.543
copy head	3.811 / 3.819	**4.126 / 4.137**	2.958 / 2.963	5.487 / 5.505
copy head + plain CE (ours)	**3.787 / 3.783**	3.928 / 3.926	2.592 / 2.587	5.485 / 5.486

Table 12. The same pattern as at 77M–305M: the LM head delegates the tokens already in the context (seen: $+0.42$) and is slightly better on new ones; the extra term removes the deficit (within 0.005) and also improves the full model. Seeds 0 / 1.

The circuit follows the same pattern. On the induction probe of Appendix G.7 (attention to the token after the previous occurrence on a repeated random sequence, the same probe as for the 77M models), the best head is 0.404 / 0.461 in the vanilla model (seeds 0 / 1, logged at the end of training), 0.127 / 0.089 with the copy head and 0.415 / 0.343 with the extra term, and the LM head's copying gain is 0.36 / 0.44 and 2.68 / 2.22 nats without and with the term. At this size the copy head blocks the circuit more strongly than at 305M (to 20–31% of the vanilla head). We predicted before the second seed that the term would restore more than 80% of the vanilla head; it restores 103% and 74%, so the restoration is partial at one seed.

H.5. Frozen experts

	full model	LM alone
LM trained alone	4.261	4.256
LM mixed with the frozen small LM (2 seeds)	5.139 / 5.139	**9.293 / 9.125**
same + plain CE (ours)	4.285 / 4.256	4.283 / 4.254
LM mixed with a frozen *tiny* LM (2 layers, d=128, loss 6.284)	5.875 / 5.824	**6.962 / 6.479**
same + plain CE (ours)	4.264 / 4.280	4.260 / 4.276

Table 13. With an expert that is better than the LM at initialisation, the LM's responsibility r starts near zero and the LM never learns: the mixture stays at the small model's loss and the LM alone ends at 9.1–9.3 nats (chance 10.8; 4.26 trained alone). The plain CE term restores normal learning.

H.6. Simple alternatives to the extra term

Why not attach the copy head afterwards? The simplest way to avoid delegation is to train the LM alone and then fit the copy head and gate on top of the frozen LM. Doing so on the plain 77M LM (100M tokens, 4.261) reaches 4.170 after 20M and 4.166 / 4.166 after 100M further tokens (two seeds), a gain of 0.09 nats; training the two jointly with the extra term reaches 3.984 in the same 100M tokens, a gain of 0.28. Most of the benefit comes from training the LM together with the copy head, which is what the mixture loss alone turns into delegation. A second simple remedy is helper dropout: train the LM alone on a random half of the micro-batches and the mixture on the rest. It reduces the delegation (LM alone 4.261 / 4.348 for two seeds, against 4.261 vanilla and 4.381 without it) but trains the copy head half as often, and the full model ends at 4.027 / 4.079; the extra term gives 3.979–3.989 with an LM head of 4.185–4.200, better on both counts (77M, 100M tokens).

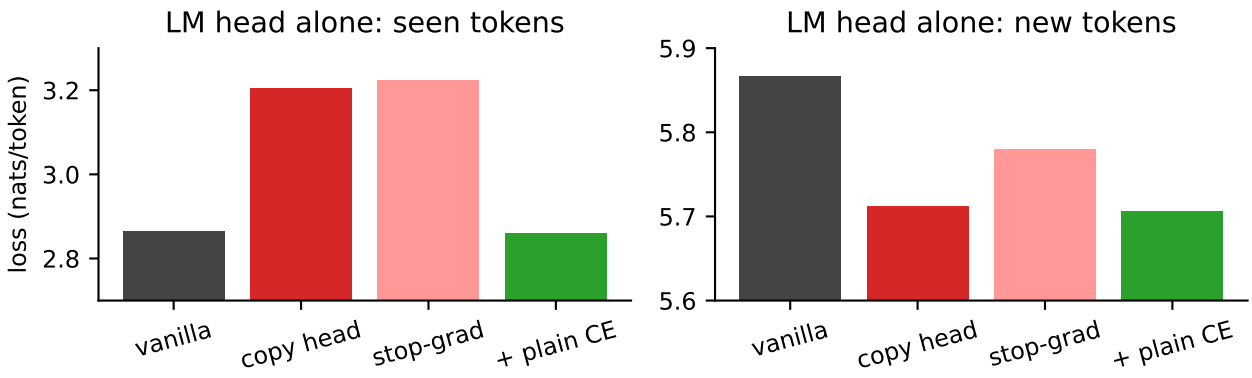


Figure 6. LM head alone (77M, 100M tokens). The copy-head model gives up seen tokens; adding the plain CE recovers them and keeps the gain on new tokens.

H.7. Downstream evaluation

	LAMBADA acc.			HellaSwag acc.		
size	vanilla	CPLM	CPLM, LM alone	vanilla	CPLM	CPLM, LM alone
135M	31.4	33.5	7.8	39.1	38.4	36.9
360M	37.2	40.4	12.2	41.8	41.8	40.9
1.7B	51.2	53.7	22.7	52.1	51.6	50.8

Table 14. LAMBADA (OpenAI test set, 5,153 passages) needs a word from the context; HellaSwag (2,000 validation items, s.e. ≈1.1) barely does. The copy head adds 2–3 LAMBADA points; without it the LM loses 23–29.

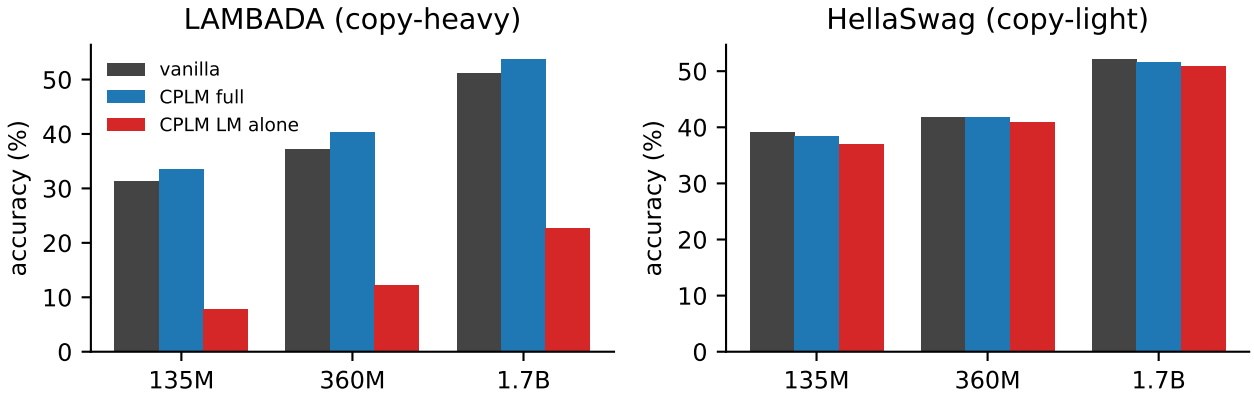


Figure 7. Downstream accuracy of the released models with and without the copy head.

H.8. Products of experts

Products delegate too. One might expect a product of experts, $p \propto p_{\text{LM}}\, p_{\text{copy}}^{\beta}$, to avoid the problem, since the LM's gradient is no longer scaled by r. It does not: the LM is now fitted to the residual $p/p_{\text{copy}}^{\beta}$ and gives up copyable tokens in logit space instead. With the same learned copy head combined by a product (77M, 100M tokens) the full model reaches 4.084 and the LM head alone 4.712, a delegation of $+0.45$ against $+0.12$ for the mixture. Whatever the combination rule, a component that is not itself trained on the plain likelihood leaves to the auxiliary whatever the auxiliary already explains. Adding the plain CE of the LM's own logits to the product loss removes it as well: LM head alone 4.244 (better than the plain Transformer's 4.261) for a full model of 4.094. Rescorla–Wagner also predicts conditioned inhibition, which only

a product allows: the LM could learn to push down the context tokens the helper boosts. Relative to the plain model it does, and only in the product; we had predicted the stronger form, context tokens pushed below tokens absent from the context, and that does not occur. On validation positions $t \geq 512$ we compare the LM head's mean log-probability of in-context non-target tokens with that of tokens absent from the context: the gap is 3.05 nats for the plain model, 3.09 for the mixture-trained head (which delegates only the target: -4.43 against -4.28) and 2.62 for the product-trained head (2.99 with the extra term); a second seed, predicted before the run to fall more than 0.2 below the plain model and to recover within 0.15 with the extra term, gives 3.07, 2.63 and 3.01. A product helper that raises every context token takes the LM's whole context prior, not only the copied target.

H.9. Repairing released models

Repairing released models. We fine-tune CPLM checkpoints for 2,000 steps (16M FineWeb-edu tokens, AdamW 3×10^{-5}) with the original loss (control) or with the added term.

	loss		LAMBADA	
	full	LM alone	full	LM alone
135M, original loss	2.751	2.920	33.2	8.3
135M, + plain CE	2.760	2.857	34.6	14.0
360M, original loss	2.569	2.698	40.2	12.2
360M, + plain CE	2.577	2.643	40.8	21.5

Table 15. Fine-tuning with the original loss leaves the delegation in place; the added term recovers 5.7 and 9.3 LAMBADA points of the LM head alone and does not hurt the full model.

Repair saturates: fine-tuning CPLM-135M with the added term for 10,000 steps (80M tokens) reaches 2.841 loss and 15.6% LAMBADA for the LM head alone (2,000 steps: 2.857, 14.0%; vanilla: 2.791, 31.4%). Delegation learned in pretraining is only partly reversible; the term should be used from the start, where it removes the delegation entirely (Table 11).

H.10. Symmetric mixtures

Where the fix does not apply: symmetric mixtures. A mixture of softmaxes (Yang et al., 2018) (4 components, same 77M LM, 100M tokens, sequence 512) shows the same lock-in: the mixture reaches 4.402 / 4.385 (two seeds), *worse* than the plain LM (4.261), and each component alone is a poor LM (5.10–7.41) with skewed responsibilities (largest 0.39 / 0.41). Intermediate weights of the per-component term (0.05, 0.2) balance the responsibilities but leave the mixture at 4.42–4.43. Adding the plain CE of every component removes the lock-in but makes the components identical (each 4.423; mixture 4.451): with no designated main model, forcing every component to model everything also removes the diversity the mixture exists for. The fix is for the asymmetric case—a main LM plus a cheaper auxiliary expert—which covers copy heads, caches and frozen experts.

I. Additional blocking results

LM head alone − vanilla (nats)	$c_y = 0$	$(0, .1)$	$[.1, .5)$	$[.5, .9)$	$[.9, .99)$	$\geq .99$
CPLM-135M	-0.047	$+0.065$	$+0.342$	$+0.616$	$+0.723$	$+0.791$
CPLM-360M	-0.052	$+0.050$	$+0.273$	$+0.467$	$+0.558$	$+0.504$

Table 16. Graded blocking in the released 135M and 360M models (20B tokens), over the bins of Table 17.

I.1. Graded blocking over the 1.7B run

LM head alone − vanilla (nats)	$c_y = 0$	$(0, .1)$	$[.1, .5)$	$[.5, .9)$	$[.9, .99)$	$\geq .99$
share of tokens (42B)	43.6%	19.0%	14.3%	11.0%	5.5%	6.5%
42B tokens	−0.059	+0.052	+0.221	+0.306	+0.361	+0.379
126B tokens	−0.054	+0.044	+0.208	+0.278	+0.340	+0.360
200B tokens	−0.057	+0.034	+0.206	+0.268	+0.321	+0.327

Table 17. Delegation rises monotonically with the copy head's reliability on the token at every checkpoint, and where the copy head cannot help ($c_y = 0$) the LM head is better than the vanilla model's: blocking in proportion to the helper's prediction, and niche partitioning. The gaps shrink slowly over training in every bin, the most reliable bin by the most in absolute terms; we had predicted it would shrink least.

I.2. Demonstrations as a helper

We fine-tune SmolLM-135M (and 360M) to label review sentences built from positive and negative adjectives with two arbitrary label words ('zebra' for positive, 'piano' for negative); test sentences use held-out adjectives. Every training question is preceded by K labelled demonstrations (*demo*), by none (*plain*), or by K demonstrations half of the time (*fix*); 300 steps of 8 questions, AdamW at 5×10^{-5}. Accuracy is measured without demonstrations (zero-shot) and with K of them, as whether the true label word has the higher log-likelihood (chance 50%).

training ($K = 4$ unless stated)	zero-shot	with demonstrations
plain, three seeds	96–97%	54–57%
demo, three seeds	**57%, 59%, 100%**	100%
fix, three seeds	89–94%	100%
demo, $K = 1/2/4/8$, six seeds each: seeds at or below 65%	0/6, 4/6, 3/6, 4/6	100%
360M: demo ($K = 2$), six seeds / fix, two / plain, two	**53–82%** / 100% / 96–98%	100%
plain for 100 steps, then demo ($K = 2$), eleven seeds	both kept in 10 of 11	100%

Logged every 25 steps ($K = 2$, six seeds), accuracy with demonstrations reaches 99–100% by step 25 in every seed, and zero-shot accuracy then stops where it is (50% in four seeds, 67% and 77% in two, fixed from about step 50 to the end); with $K = 1$ it rises further before freezing (73–94%). This is the freeze of the blocked component once the shared error is gone, with the in-context route as the helper. In the one seed of eleven where the in-weights skill was erased after it had been learned (93% at step 100, 55% from step 150 on), the demonstrations reached 100% within 25 steps of their introduction. Several of our predictions failed: blocking does not rise monotonically with K beyond two demonstrations; freezing the attention layers does not prevent it (zero-shot 51–55% with demonstrations still read at 100%, since the pretrained model's induction heads copy the label without further training); the invasion criterion of Proposition 2, read on accuracy, does not predict the K threshold, since in-context accuracy saturates within 25 steps at every K; and erasure after zero-shot training was predicted in at least two of eight further seeds and occurred in none. The reverse effect, fine-tuning without demonstrations degrading in-context learning (our plain rows), has been reported before; mixing training formats is common practice in instruction tuning; what blocking adds is why demonstrations alone prevent the in-weights skill and when it freezes.

I.3. Retrieval blocking in released LMs

closed-book accuracy	A (trained with passage)	B (never)
released 135M: start	2.4%	2.8%
released 135M: passages always present (800–2,000 steps)	**2.4%**	100%
released 135M: passage dropped from 2% of A questions (1,200 steps)	19.6%	98.8%
released 135M: + each A question also trained closed-book (ours)	100%	100%
released 360M: start	2.0%	2.0%
released 360M: passages always present (800–1,200 steps)	**1.6%**	100%
released 360M: + each A question also trained closed-book (ours)	100%	100%
released 1.7B, LoRA rank 16: start	2.4%	2.8%
released 1.7B, LoRA: passages always present (800–1,200 steps)	**2.0%**	100%
released 1.7B, LoRA: + each A question also trained closed-book (ours)	100%	100%

Table 18. The released LM stores none of the facts it was always given in a passage: closed-book accuracy stays at chance while the other half is learned perfectly. Unlike the small synthetic model, 2% passage dropout is not enough here within 1,200 steps (each fact is seen without its passage about once); training every question closed-book as well is.

I.4. Released LM with a frozen expert

135M, +8M tokens (two data slices)	LM alone	seen	new	mixture
released checkpoint (start)	2.791	1.729	4.189	–
continued plain	2.787/2.786	1.730/1.728	4.179/4.180	–
continued mixed with the frozen 360M	**2.866/2.868**	1.758/1.756	4.326/4.332	2.588/2.588
same + plain CE (ours)	2.787/2.787	1.729/1.727	4.181/4.183	2.590/2.590

Table 19. The gate gives the expert 86% of the mass, and the released LM ends worse than it started ($+0.075/+0.077$ nats). With the extra term it matches plain training while the mixture is as good as without it.

I.5. Partial calculator

LM-alone answer-digit accuracy (seeds 0/1/2)	all problems	covered by the tool	not covered
trained alone	100/100/100	100/100/100	100/100/100
mixed with the partial calculator	63/63/66	**33/34/38**	100/100/100
same + plain CE (ours)	100/100/100	100/100/100	100/100/100
no tool, covered answers removed from the loss	36/65/63	**32/36/34**	41/100/100

Table 20. 4,000 steps. Mixed with a tool that covers part of the input space, the LM learns to add with carries perfectly and fails at the easier problems without a carry, which the tool covers. The last row tests Proposition 7: removing the covered answers from the loss, with no tool, gives the same covered accuracy (seed 0 had not finished learning the uncovered problems at 4,000 steps).

Proposition 7 (Blocking is complement training)**.** *Let a helper be right on a set C of positions, with gate $\lambda \geq \lambda_0$ there and 0 elsewhere, and let $\theta(t)$ follow gradient flow on the mixture loss and $\tilde{\theta}(t)$, from the same start, on the plain loss with the positions in C removed. If $\|\nabla_\theta \log q_y\| \leq G$ and the masked gradient is L-Lipschitz, then $\|\theta(t) - \tilde{\theta}(t)\| \leq \frac{\pi_C G}{L} \frac{1-\lambda_0}{\lambda_0} (e^{Lt} - 1)$, with π_C the weight of C in the loss.*

Proof. Outside C the two losses agree. On C the mixture gradient is $r\nabla(-\log q_y)$ with $r = (1-\lambda)q_y/p_y \leq (1-\lambda_0)/\lambda_0$, so the two vector fields differ by at most $\pi_C G(1-\lambda_0)/\lambda_0$; Grönwall's inequality gives the bound. □

As the gate comes to trust the helper, the LM's competence on the covered inputs is therefore what it would transfer from the rest of the data alone; the 33–38% of the blocked LM is that transfer, not a residue of the covered examples. The experiment departs from the proposition's assumptions (Adam, a learned gate) and still matches it. In a language model the proposition gives a bound rather than a prediction, because a real copy head covers only part of the seen tokens. A 77M

LM trained for 100M tokens with every seen-target position removed from the loss has 7.231 nats on seen tokens and 5.596 on new ones, against 2.885 and 5.950 for the plain model. The copy-augmented LM head (no extra term) sits at 3.200 and 5.768: it pays 7% of the bound's cost on seen tokens and collects 51% of its capacity gain on new ones; with the extra term it pays nothing (2.862) and keeps the gain (5.755). Why the extra-term model is better than the plain one even on new tokens, where it is not relieved, follows in part from Proposition 5: the copy head sends credit into the backbone it reads. Stopping that gradient (extra term kept) gives 2.948 and 5.842: about 45% of the new-token gain and all of the seen-token advantage go with it. The extra term also reweights: the LM's gradient weight is $1+r$, about 2 where the copy head cannot help and 1 where it can. A plain Transformer with weight 2 on new-target positions and 1 on seen ones reaches the same new-token loss (5.748) but pays on seen tokens (3.116; overall 4.332 against 4.300). Reweighting is enough for the size of the new-token gain; credit through the copy path is what lets the copy-augmented model take it without the seen-token cost. The weighting test is cruder than $1 + r$ (it gives weight 1 to every seen position), so the two shares do not separate exactly.

I.6. Calculator

answer-digit accuracy of the LM alone (tool removed)	seed 0	seed 1	seed 2
trained alone	100%	–	–
trained mixed with the calculator	**1.7%**	**0.0%**	**0.0%**
same + plain CE (ours)	100%	100%	75.7%
own errors weighted by the gate (Jacobs et al., 1991)	6.2%	6.8%	0.0%
mixture + LM's own error weighted by $1 - \lambda$	0.0%	0.0%	2.5%

Table 21. 4,000 steps of 64 sequences. With the tool in the likelihood the LM never learns to add; with the extra term it learns as well as alone (seed 2 still improving at the end of training). Own-error terms weighted by the gate, as in the competitive objective of Jacobs et al. (1991), do not help: once the gate trusts the tool, the LM's weight $1 - \lambda$ vanishes and it is blocked again. The fix works because its weight does not depend on the gate.

I.7. Invasion phase diagram

helper reliability ρ	0.5	0.7	0.9	0.97	1.0
LM at $q = 0.999$: final gate / covered accuracy	0.00 / 100%	0.00 / 100%	0.00 / 100%	0.00 / 100%	**1.00 / 87%**
LM at $q = 0.953$: final gate / covered accuracy	0.00 / 100%	0.00 / 100%	0.00 / 100%	0.00 / 100%	**1.00 / 33%**
seed 1, LM at $q = 0.999$: final gate	0.00	0.00	0.00	**0.66**	**1.00**
seed 1, LM at $q = 0.937$: final gate	0.00	**0.40**	0.05	0.02	**1.00**
seed 2, LM at $q = 0.999$: final gate	0.02	0.00	0.00	0.01	**1.00**
seed 2, LM at $q = 0.683$: final gate	0.00	0.00	0.00	**0.65**	**1.00**

Table 22. Only a perfect helper invades and erases; every imperfect one is excluded. At $\rho = 0.97 > q = 0.953$ the proposition predicts invasion, which did not occur: the LM keeps learning after attachment and passes ρ before the gate grows. The criterion holds at the moment of attachment, not over a race. At seeds 1 and 2 (lower rows; the LM's competence at attachment differs by seed) three imperfect helpers invade partly, two of them with $\rho < q$: the proposition's local criterion does not decide every run.

Trained from scratch with an imperfect helper, the LM alone reaches about 100% on the covered problems after 750, 1,750 and 3,250 steps for $\rho = 0.5, 0.8, 0.9$, and not within 4,000 steps for $\rho = 0.95$ (about $350/(1-\rho)$; Figure 2a). We predicted that two more seeds would land within 50% of $350/(1-\rho)$; they give 1,000 and 1,250, 1,000 and 1,750, 1,750 and 2,250, and 1,750 and not within 4,000 steps, so the order holds for $\rho \leq 0.9$ in one seed and roughly in the other, and the law holds only within a factor of two (it fails at $\rho = 0.5$ in seed 2 and $\rho = 0.95$ in seed 1).

I.8. Erasure in a 77M LM

LM-head loss (seeds 0/1)	seen in context	new
starting checkpoint (plain, 100M tokens)	2.656	5.623
+100M tokens, plain	2.590/2.583	5.552/5.558
+100M tokens, copy head attached	**2.876/2.870**	5.481/5.483
+100M tokens, copy head + plain CE (ours)	2.637/2.631	5.514/5.517

Table 23. Attaching a copy head to a trained LM makes its LM head worse than it was at the start on exactly the tokens the copy head can supply ($+0.22$ nats), while it improves on the others beyond plain training. With the extra term the seen-token loss does not rise. Full-model losses (seed 0): copy head 3.819, copy head + plain CE 3.835, plain 3.963.

I.9. Reinforcement-learning reading, hysteresis and relearning

Read as reinforcement learning, this loss is the log of the expected reward of a one-step policy that may answer itself or call a free, always-correct tool: $\log\big(\pi(y) + \pi(\textsc{call})\big)$. The extra term is then a counterfactual objective for the policy's own answer, trained whether or not it called the tool. With the linear expected reward instead, the LM alone does not learn the task (30%, the trivial leading digit, after 6,000 REINFORCE steps; likewise after 1,000 steps of exact expected-reward ascent), so we do not report that variant. A dynamical model of human learners with tools predicts the same collapse of competence under reliance (Krakauer, 2026); here the mechanism is exact and follows from the training objective. That model also predicts hysteresis and bistability. We see the first: switching the extra term on after 2,000 steps of mixed training, the LM reaches 46%, 58% and 72% after 500, 1,000 and 2,000 further steps, while with the term from the start it reaches 100% within 1,000 steps. We do not see the second: for small weights of the term the LM ends at intermediate accuracies that vary continuously across seeds (weight 0.01: 27–61%, five seeds), not in two separated states. Attention theories of conditioning (Mackintosh, 1975) predict a further effect that Rescorla–Wagner does not: a blocked cue learns more slowly even after the blocking cue is gone. Removing the calculator after 2,000 steps of mixed training and continuing alone, the LM reaches 66/63/69% after 1,000 steps against 100/100/99% for a fresh model, but a control trained for the same 2,000 steps with the answer digits excluded from the loss is also slow (82/96/64%), so we do not attribute the slowdown to blocking.

I.10. Erasure in the calculator task

A tool erases a skill the LM already has. We pretrain the LM alone for 4,000 steps (100% on every problem), then continue for 6,000 steps on the same data, now mixed with the partial calculator. Nothing about the data changes.

after 4,000 + 6,000 steps (seeds 0/1/2)	covered by the tool	not covered
continued alone	100/100/100	100/100/100
continued mixed with the partial calculator	**36/34/35**	100/100/100
continued mixed + plain CE (ours)	100/100/100	100/100/100

Table 24. A competent LM loses the covered skill once the tool is in its likelihood. The collapse is abrupt: covered accuracy stays above 87% for 1,000–4,000 steps and then falls to the from-scratch level within about 1,000 steps (seed 0: 100, 100, 98, 93, 87, 43, 36% at steps 4k–10k; seed 1: 100, 100, 35%; seed 2: 100, 99, 97, 92, 90, 35%).

This is forgetting without distribution shift: the data are unchanged, only the likelihood now credits the tool. The LM ends at the same 33–38% it reaches when it never learned the skill (Table 20), so the end state does not depend on the starting competence. With the extra term the skill is kept on all three seeds. The gate gives the tool $>$99% of the mass on covered digits within 200 steps of attaching it, long before the collapse, so the collapse is not a tipping point of the gate; with the LM's gradient on covered digits near zero, the skill drifts under the updates driven by the rest of the data. The drift depends on the step size: lowering the learning rate tenfold after attaching the tool (or raising Adam's ϵ to 10^{-2}, which shrinks the same steps) keeps 98–100% on covered problems for the 6,000 steps we run; we cannot say from these runs whether it delays the erasure or prevents it.

I.11. What sets the erasure floor

In the shared-vocabulary task the erased skill settles at 33–38%, not at chance. Under the mixture loss a blocked target receives almost no gradient where it is covered, but its logit is still pushed down by the softmax normaliser at every other position, and pushed up wherever the same token is an unblocked target. We predicted that moving the covered answers onto their own token ids (digits 20–29, used only in covered problems) would remove that downward pressure from the shared digits and weaken erasure. It did the opposite: with disjoint ids the covered skill is erased completely (0% / 0% at 10k steps, two seeds, against 36% / 34% with shared ids). The result fits the normaliser account read the other way: a token that is a target only where the helper covers it receives only downward pressure and dies, while shared digits stay alive through the positions where they are still learned, which sets the floor. We report the prediction as failed and this reading as post hoc. We then tested the reading with a prediction fixed before the run: if the disjoint ids are also used for half of the uncovered problems, they are again learned somewhere, and the covered skill should keep a floor above zero. It does in both seeds (13% and 30% at 10k steps, fluctuating between 5% and 33%), below the shared-vocabulary floor and noisier, but not zero.

I.12. Repetition within and across documents

If later mentions of a fact in one document can be copied from the first, they should teach the weights less than the same mentions spread over separate documents. We fine-tune the released 135M baseline on documents about 400 fictional people (learning rate 2×10^{-4}, one run) with four paraphrased mentions of each person's birthplace either in one document or in four documents (the same tokens), or a single mention, and probe "… was born in" without context (chance 2%). Closed-book accuracy at epochs 5/10/15/20: within one document 4.5/14/48/98.8%; across four documents 7/22/99.5/100%; single mention 1.8/3.7/8.7% (that run stopped at epoch 15 on a GPU fault). Repetition within a document is less effective than across documents, as blocking predicts, but far from useless: we had predicted it would match a single mention, and it does not (the single-mention run also takes a quarter of the optimiser steps).

I.13. A weak helper

A weak helper does little. Attaching a neural cache (Grave et al., 2017) (a parameter-free copy distribution over the context, with a learned per-token gate) to the released 135M baseline of Godey and Artzi (2026) and continuing for 8M tokens, the cache adds only 0.004 nats to the mixture and the gate settles near 1% mass; the LM head's seen-token loss moves by the same order (1.730 plain, 1.734 with the cache, 1.731 with the extra term; new tokens 4.179, 4.175, 4.177). Blocking scales with how much the helper explains, as the shared-error rule says.

I.14. Retrieval blocking in a synthetic task

Retrieval in the context blocks memorisation. The helper need not be a separate distribution. We train the same small Transformer on 2,000 name→value facts asked as questions. For half of the facts (set A) every training sequence also contains a 'retrieved' passage stating the fact earlier in the context; the other half (B) never has one. We then ask every fact closed-book. In Kamin's design the blocking cue is made competent first, so we first train 2,000 steps of pure copying (passages with fresh random values, nothing to memorise), then 3,000 steps on the facts.

closed-book recall (seeds 0/1/2)	A (trained with passage)	B (never)
copy pretraining, then facts with passages	**23/3/13%**	100/100/100%
same + each question also trained closed-book (ours)	100/100/100%	100/100/100%
copy pretraining, then facts without passages	100/100/100%	100/100/100%
no copy pretraining, facts with passages (5,000 steps)	100/100/100%	100/100/100%

Table 25. Once the model can read the answer from its context, facts that are always supplied there are not stored. Without copy pretraining the facts are memorised before copying is learned, and nothing is blocked: the order dependence of blocking.

There is no mixture here: the passage and the weights feed one network, and the error is shared through it. Blocking needs the helper to be competent first, as in Kamin's design; when memorisation wins the race (last row), the facts are stored even though the passages are present. The extra term is the closed-book version of each question, the analogue of training the LM head without the helper. A small dose suffices here: dropping the passage from a random 2% of the set-A

questions during training restores closed-book recall to 99.9/100% (5% and 10%: 100% on both seeds), so in this task retrieval dropout of a few percent removes the blocking.

I.15. Distractors, paraphrases and secrets

With two distractor passages about other people around the relevant one (135M, 1,200 steps), blocking is unchanged: closed-book accuracy 0.8% for set A against 96% for set B, and with a paraphrased question ("What is the birthplace of . . . ?") 2.4% against 52%. With the extra term both sets reach 100% on the trained wording and the same level on the paraphrase (39% and 36%): the facts are stored as well as facts fine-tuned without passages, whose paraphrase transfer is itself limited.

How complete blocking is: training on secrets without storing them. Blocking can be used deliberately. The facts of set A received the full loss on every training step, yet after fine-tuning (135M, 1,000 steps) the model answers them from a passage perfectly (open-book 100%) and not at all without one (closed-book 2.4%, paraphrased 1.6%, chance 2%). Supplying a sensitive span in the context while training on it thus keeps it out of the weights even though every token is trained: blocking, not omission. We report this as a demonstration of how complete blocking is, not as a recommended defence: when the span is known, simply removing it from the loss does as well against our attacks at lower cost (synthetic records, seed 0: extraction 0%, mean rank 52.0 of 100, held-out loss 0.564 against 0.68–0.78 with the card and 0.54 plain). We test this on documents about 400 fictional people, each with a 10-digit phone number, fine-tuning the released 135M model on the documents (1,000 steps of 16, learning rate 2×10^{-4}). Memorisation is greedy extraction and the teacher-forced accuracy on the number's tokens after "… can be reached at" with no card, for training people against held-out ones.

released 135M (seeds 0/1/2)	exact extraction	number tokens, train	held out	held-out loss, other
plain fine-tuning	100/100/100%	100/100/100%	31/31/30%	.540/.576/.533
goldfish loss (Hans et al., 2024)	2.5/3.0/3.5%	85/85/85%	31/31/31%	.561/.562/.567
number card in context, all tokens trained	**0/0/0%**	**27/27/26%**	28/28/26%	.687/.679/.690
number removed from the loss (seed 0)	0%	22%	23%	.564
name and number card in context (seed 0)	–	27%	27%	1.342

Table 26. Supplying the number in the context while training on it leaves no trace of it in the weights (train equals held out), where plain training memorises every number and goldfish loss most of them; the price is 0.15 nats on the rest of the held-out documents, and much more if the card also supplies the name. Goldfish loss drops every 4th token from the loss.

Exact extraction is greedy decoding of the number for the first 200 training people. Goldfish loss stops the exact number from coming out but leaves 85% of its tokens in the weights; the card leaves neither. A ranking attack is stronger: ranking each true number against 99 random numbers of the same format by likelihood (Carlini et al., 2019) (seed 0, rerun), plain fine-tuning puts the true number first for 100% of training people and goldfish loss for 99.5% (mean rank 1.0 for both), so goldfish loss hides the number from greedy decoding but not from the model's likelihood. With the card the mean rank is 52.9 of 100 with no number ranked first, the same as for held-out people (51.3). An attacker who knows the card format and ranks the numbers after the card's own prefix ("[Record] phone:") does no better (mean rank 54.0, 1% ranked first): the card itself carried no loss, and in the document, where the number did carry the full loss, it was blocked by the copy from the card. The utility cost of the card was larger in this rerun (0.784; 0.68–0.69 in the three runs above). The released 360M model behaves the same (learning rate 10^{-4}): plain fine-tuning extracts 100% of the numbers and ranks every one first (mean rank 1.0), while with the card extraction is 0%, number-token accuracy is 24.9% against 24.7% for held-out people, and the mean rank is 52.2 (held out 50.1), at a held-out loss of 0.758 against 0.585.

Realistic records. We repeat the test on 400 English records of the ai4privacy PII-masking corpus (ai4privacy, 2024), each containing one high-risk secret (an e-mail address, password, card, account or phone number, or IPv4 address) in realistic text, with 80 held-out records (135M, 1,000 steps of 16, learning rate 2×10^{-4}, seed 0). The attacks are given the record's own text up to the secret; the ranking attack ranks the true secret against 99 real values of the same type from other records.

ai4privacy records, released 135M	extraction	token acc. (train / held out)	rank (mean, top-1)	held-out loss
plain fine-tuning	100%	100% / 20%	1.0, 100%	6.84
goldfish loss	1%	80% / 19%	1.2, 94%	6.88
secret supplied in context	**0%**	18% / 14%	**49.8, 2.5%**	6.36
every PII value of the record supplied	**0%**	16% / 14%	**48.3, 1.5%**	**5.75**
lighter fine-tuning: 250 steps, learning rate 10^{-4}				
plain fine-tuning	59%	92% / 20%	1.1, 97%	6.07
goldfish loss	1.5%	76% / 19%	1.4, 92%	6.25
secret supplied in context	**0%**	18% / 17%	**50.6, 2.5%**	5.03
every PII value of the record supplied	**0%**	16% / 15%	**51.4, 1.5%**	**4.65**

Table 27. On realistic records plain fine-tuning and goldfish loss both leave the secret ranked first for nearly every training record; with the secret in the context it ranks like a random value. Held-out losses are high for all four because 1,000 steps over 400 short records overfit (the model scores 4.39 before fine-tuning); no fine-tuned model beats the starting model on held-out text here, so the last column measures overfitting, not useful learning, and we draw no utility conclusion from it; DP-SGD with noise $\sigma \geq 0.5$ also leaks nothing here, and also barely moves the model (4.32–4.35). One run each.

J. Negative results on architecture changes

Each against a same-size control (small scale, two seeds unless noted): cache read-back into the final state (-0.013 small, gone at 77M); mid-layer read of next-token embeddings (worse than the same module with hidden-state values); two-pass hindsight attention (0 at $2\times$ compute); delta values $v_t - v_{t-1}$ (0); vocabulary-thought layer (0); revision attention (equal to baseline); FFN experts routed by in-context repetition ($+0.14$); semantic copy expert ($+0.02$); max-normalised combination (0); product-of-experts copy (-0.011 at $+26\%$ time).